

EduBridge: Design and Development of an Online Learning and Teaching Platform with AI Assistance

Raufu Umme Rhidi

ID:22103286

A Practicum in the Partial Fulfillment of the Requirements
for the Award of Bachelor of Computer Science and Engineering (BCSE)



Department of Computer Science and Engineering
IUBAT School of Computer Science & Engineering
IUBAT—International University of Business Agriculture and Technology

Spring 2026

EduBridge: Design and Development of an Online Learning and Teaching Platform with AI Assistance

Raufu Umme Rhidi

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of
Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

Prof. Dr. Utpal Kanti Das
Dean, ISCSE
Chairman, IUBAT

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

Drs. Ferdaus Anam Jibon
Supervisor and Associate Professor

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2026

Letter of Transmittal

10 May 2026

The Chair

Practicum Defense Committee

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

4 Embankment Drive Road, Sector 10, Uttara Model Town

Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

I am writing to formally submit my practicum report, "EduBridge: Design and Development of an Online Learning and Teaching Platform with AI Assistance" in partial fulfillment of the requirements for the BCSE degree. This document details the software development life cycle, system architecture, and successful implementation of the platform. Developing EduBridge was an invaluable opportunity to bridge theoretical computer science with practical software engineering, particularly through the integration of AI and secure financial systems. I sincerely thank my supervisor and the faculty for their guidance, and I hope this report meets the expectations of the Defense Committee.

Yours sincerely,

Raufu Umme Rhidi

ID: 22103286

Organization's Certificate



Student's Declaration

I am Raufu Umme Rhidi, ID#22103286, a student of the BCSE – Bachelor of Computer Science and Engineering program under the College of Engineering and Technology (CEAT) at IUBAT – International University of Business Agriculture and Technology. I hereby declare that this practicum report titled " EduBridge: Design and Development of an Online Learning and Teaching Platform with AI Assistance " submitted to the Department of Computer Science and Engineering at IUBAT is my original work. I confirm that this project has not been previously submitted for any other degree or qualification.

Furthermore, I assure that there is no plagiarism or data falsification in this report. All materials, concepts, and source codes utilized during the development of this platform have been properly acknowledged. Any information derived from external sources, including academic journals, technical documentation, and third has been duly cited in the references section in accordance with strict academic standards,

Raufu Umme Rhidi
ID: 22103286

Supervisor's Certification

This is to certify that the practicum report entitled “EduBridge: Design and Development of an Online Learning and Teaching Platform with AI Assistance” has been carried out by Raufu Umme Rhidi, ID#22103286, a student of the BCSE program at IUBAT—International University of Business Agriculture and Technology, as partial fulfillment of the requirements of the Practicum Defense course (CSC 490).

The report has been prepared under my supervision and represents a true record of the work successfully accomplished by the student. To the best of my knowledge, and as per the student's declaration, no part of this report has been submitted elsewhere for any degree, diploma, or certification.

The student is hereby permitted to submit the report for evaluation. I wish them every success in their academic pursuits and future professional endeavors.

Drs. Ferdaus Anam Jibon

Associate Professor

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Abstract

The rapid expansion of digital education has created a high demand for interactive online learning environments. However, many existing platforms lack instant academic support and transparent financial management for educators. To address these issues, this practicum project details the development of EduBridge, a comprehensive online learning and teaching platform designed to connect expert instructors with students worldwide. The system was developed using the Agile Scrum methodology. Its technical architecture features a secure backend powered by the Laravel PHP framework and MySQL database, alongside an intuitive and highly responsive frontend built utilizing Tailwind CSS. A core innovation of EduBridge is its integrated Artificial Intelligence tutor. By leveraging the Groq API and Retrieval Augmented Generation techniques, the platform provides students with immediate, context aware answers. These responses are based strictly on instructor provided lesson transcripts, which prevents hallucinatory outputs and ensures high academic accuracy. Additionally, the platform introduces an automated financial module integrating Stripe and SSLCommerz payment gateways. This system instantly divides course sales revenue, automatically allocating seventy percent to the instructor and thirty percent to the platform administration. This transparent process eliminates manual payout delays and builds trust among stakeholders. Following extensive functional and security testing, EduBridge successfully demonstrates how modern web technologies and artificial intelligence can be combined. The project provides a scalable, secure, and economically sustainable solution that empowers both learners and independent educators in the digital age.

Keywords: Online Learning Platform, Retrieval Augmented Generation RAG, AI Tutor, Educational Technology EdTech, Payment Gateway Integration, Automated Financial Splitting, Agile Software Development.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to the Almighty for giving me the strength, patience, and capability to successfully complete this practicum project and report.

I am profoundly thankful to my supervisor, Drs. Ferdaus Anam Jibon, Associate Professor, Department of Computer Science and Engineering, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the development of the EduBridge platform. His technical insights and mentorship were instrumental in shaping the architecture of this project and helping me overcome various software engineering challenges.

I also extend my sincere appreciation to Prof. Dr. Utpal Kanti Das, Dean and Chairman of the Department of Computer Science and Engineering, for his support and for providing the necessary facilities at IUBAT. I am grateful to the entire faculty of the Department of Computer Science and Engineering for providing a robust academic foundation and a highly supportive learning environment throughout my studies.

Finally, I would like to thank my family and friends for their unwavering moral support, understanding, and motivation during the long hours of development and writing. This achievement would not have been possible without their continuous belief in my abilities.

Table of Contents

Letter of Transmittal	iii
Organization's Certificate.....	iv
Student's Declaration.....	v
Supervisor's Certification	vi
Abstract	vii
Acknowledgments.....	viii
List of Figures	xiv
List of Tables	xvi
Chapter 1. Introduction	1
1.1 Introduction.....	2
1.2 Background of the Study	2
1.3 Methodology	2
1.3.1 Primary Sources	3
1.3.2 Secondary Sources	3
1.4 Objectives	3
1.4.1 Broad Objective	4
1.4.2 Specific Objectives	4
1.5 Process Model	4
1.6 Feasibility Study	5
1.6.1 Technical Feasibility	5
1.6.2 Economic Feasibility	5
1.6.3 Operational Feasibility.....	6
1.7 Structure of the Report.....	6
Chapter 2. Organizational Overview	8
2.1 Organizational Overview	9

2.2 Organization Vision	9
2.3 Organization Mission.....	9
2.4 Organization Services	9
2.5 Organizational Structure	10
2.6 My Position in this Organization	11
2.7 Address of the Organization	12
Chapter 3. Requirement Engineering.....	13
3.1 Requirement Engineering	14
3.2 Requirement Elicitation	14
3.2.1 Requirement Gathering.....	15
3.3 Requirement Analysis.....	15
3.4 Requirement Specifications	17
3.4.1 User Requirements.....	18
3.4.2 System Requirements.....	18
3.4.3 Functional Requirements	21
3.4.4 Non-Functional Requirements	23
3.5 Use Case Diagram of the System.....	24
3.5.1 Functional Description of Actors.....	24
Chapter 4. Analysis	26
4.1 Software Analysis Pattern.....	27
4.2 Activity Diagram	27
4.3 Swim-lane Diagram	32
4.4 Class Diagram.....	37
Chapter 5. Project Management.....	38

5.1 Project Management	39
5.2 Risk Identification.....	39
5.3 Risk Analysis	40
5.4 Risk Planning.....	41
5.5 Risk Monitoring.....	42
5.6 RMMM Plan	43
Chapter 6. Project Planning	46
6.1 System Project Estimation	47
6.1.1 Effort Estimation (in Person-Hours).....	47
6.1.2 Cost Estimation (BDT)	48
6.1.3 Time Estimation (Calendar Weeks).....	48
6.1.4 Team Estimation	48
6.2 Function Matrix	49
6.3 Identifying Complexity	52
6.3.1 Identifying Complexity of Transition Function.....	52
6.3.2 Identifying Complexity of Data Function.....	55
6.3.3 Unadjusted Function Point Contribution (Transition Function)	56
6.3.4 Unadjusted Function Point Contribution (Data Function)....	57
6.3.5 Performance and Environmental Impact	58
6.4 Project Scheduling Chart	59
6.5 Accounts Table	60
6.5.1 Personnel Cost	60
6.5.2 Expected Hardware Cost.....	61

6.5.3 Expected Software Cost	62
6.5.4 Expected Other Cost	62
6.5.5 Total Accounts Summary	63
Chapter 7. Designing	64
7.1 Interface Design	65
7.1.1 Learner (Student) Interfaces	65
7.1.2 Instructor (Teacher) Interfaces	69
7.1.3 Administrator Interfaces	73
7.2 Data Flow Diagram (DFD)	76
7.2.1 Context Diagram (Level 0 DFD)	76
7.2.2 Level 1 Data Flow Diagram	76
7.2.3 Level 2 Data Flow Diagrams	77
7.3 Database Design	80
7.3.1 Normalization of Data Fields	81
7.3.2 Entity Relationship Model	83
Chapter 8. System Quality and Testing	85
8.1 System Quality Management	86
8.2 Quality Management Process	88
8.3 Software Test Cases	90
Chapter 9. Ethical Consideration	94
9.1 Ethical Considerations in the Software Development Process	95
9.1.1 Data Privacy and Security	95
9.2 Sustainability in the Software Development Process	96
Chapter 10. Conclusion	98

10.1 Brief Overview of the Project	99
10.2 Proposed System Benefits.....	99
10.3 Limitations of the Project.....	100
10.4 Practicum and Its Value	100
10.5 Future Plan	101
References	102

List of Figures

Figure 1.1: The Agile Scrum Process Model for EduBridge Development.	5
Figure 2.1: Organizational Structure of SourceExpert.	11
Figure 3.1: Use Case Diagram for the EduBridge Platform	25
Figure 4.1: Activity Diagram for User Registration & View User Profile	28
Figure 4.2: Activity Diagram for Add Admin & Manage Customer	28
Figure 4.3: Activity Diagram for AI Tutor Interaction & Global Announcement	29
Figure 4.4: Activity Diagram for Course Enrollment with Payment & Course Creation..	29
Figure 4.5: Activity Diagram for Course Approval& Waitlist	30
Figure 4.6: Activity Diagram for Certificate & Review	30
Figure 4.7: Activity Diagram for Instructor Application & Instructor Grading Assignments	31
Figure 4.8: Activity Diagram for Payout Request	32
Figure 4.9: Swim-lane Diagram for Authentication & Registration.....	33
Figure 4.10: Swim-lane Diagram for AI Tutor Interaction.....	33
Figure 4.11: Swim-lane Diagram for Global Broadcast	34
Figure 4.12: Swim-lane Diagram for Course Enrollment.....	34
Figure 4.13: Swim-lane Diagram for Course Creation & Approval.....	35
Figure 4.14: Swim-lane Diagram for Instructor Grading Assignments.....	35
Figure 4.15: Swim-lane Diagram for Certificate Generation & Claiming	36
Figure 4.16: Swim-lane Diagram for Payout Request	36
Figure 4.17: Class Diagram	37
Figure 6.1: Project scheduling chart	60
Figure 7.1: EduBridge public homepage and course catalog and Teacher application	66
Figure 7.2: Dedicated Course Viewer interface integrating video lessons, syllabus tracking, and the real-time AI Tutor assistant.....	67
Figure 7.3: Course Catalog and Enrollment interface featuring dynamic pricing and the 'Join Waitlist' functionality.....	68

Figure 7.4: The Learner Dashboard displaying active course progress, learning streaks, and earned certificates.	69
Figure 7.5: Instructor Creator Dashboard featuring high-level analytics, including enrollment trends and real-time revenue tracking.	71
Figure 7.6: Instructor interfaces for Grading, Course Announcements, and Earnings Management.	72
Figure 7.7: Admin interfaces for Platform Overview, Course Approvals, and Instructor Applications.	74
Figure 7.8: Admin interfaces for Global Announcements, Category Management, and Financial Payouts.	75
Figure 7.9: Level 0 Data Flow Diagram (Context Diagram) of the EduBridge Platform.	76
Figure 7.10: Level 1 Data Flow Diagram detailing core subsystems and data stores.	77
Figure 7.11: Level 2 DFD for Process 1.0 (User Management).	78
Figure 7.12: Level 2 DFD for Process 2.0 (Course Management).	78
Figure 7.13: Level 2 DFD for Process 3.0 (Enrollments & Payments).	79
Figure 7.14: Level 2 DFD for Process 4.0 (Interactive Learning & AI Tutor Workflow).	79
Figure 7.15: Level 2 DFD for Process 5.0 (Assessments & Financial Payouts).	80
Figure 7.16: Database Schema.	81
Figure 7.17: Entity Relationship Diagram (ERD) of the EduBridge Platform.	84

List of Tables

Table 5.1: Comprehensive Risk Identification	39
Table 5.2: Risk Probability and Impact Analysis	40
Table 5.3: Strategic Risk Planning.....	41
Table 5.4: The RMMM Chart.....	43
Table 6.1: Effort Estimation by Module	47
Table 6.2: Time Estimation by Project Phase	48
Table 6.3: Team Resource Allocation	49
Table 6.4: Function Matrix describing all modules and functions of the EduBridge system.	49
Table 6.5: Identifying complexity (Transition Function)	52
Table 6.6: Identifying complexity (Data Function)	55
Table 6.7: Unadjusted function point contribution (Transition Function).....	56
Table 6.8: Unadjusted function point contribution (Data Function).....	57
Table 6.9: Performance and Environmental Impact	58
Table 6.10: Estimated Development Effort Rates by Technology Stack	59
Table 6.11: Personnel Cost Estimation.....	61
Table 6.12: Expected Hardware Cost Estimation	61
Table 6.13: Expected Software Cost Estimation	62
Table 6.14: Expected Other Cost Estimation.....	62
Table 6.15: Total Accounts Summary	63
Table 7.1: Unnormalized / 1NF Course Enrollment Data	82
Table 7.2: Enrollments Junction Table (2NF)	82
Table 7.3: Students Table (2NF).....	82
Table 7.4: Courses Table with Transitive Dependencies (2NF).....	83
Table 7.5: Normalized Courses Table (3NF).....	83
Table 7.6: Normalized Instructors Table (Mapped via users table).....	83
Table 8.1: Tools and Technologies Utilized for Quality Assurance.....	89
Table 8.2: Test Case 1 - Secure Course Enrollment	90

Table 8.3: Test Case 2 - AI Tutor Contextual Accuracy (RAG)	90
Table 8.4: Test Case 3 - Instructor Revenue Payout.....	91
Table 8.5: Test Case 4 - Automated Quiz Grading.....	92
Table 8.6: Test Case 5 - Course Moderation Workflow	93

Chapter 1. Introduction

1.1 Introduction

Rapid changes in the realm of e-learning have greatly impacted ways of distributing information in today's world. This chapter offers a detailed analysis of EduBridge an online learning and teaching platform, an effective LMS that utilizes new technologies and design elements in order to meet current educational needs. With an increasing demand for distance learning, this project is centered around the creation of a connection between traditional pedagogical practices and automated solutions. This introduction serves as a starting point in analyzing web development technologies combined with AI technologies and discussing the way EduBridge an online learning and teaching platform works.

1.2 Background of the Study

In today's education scene in Bangladesh, emphasis on digital literacy and skills is prevalent. However, many of the online learning modules available today are static in nature, and mostly serve only as video archives without much interaction. There is a "guidance deficiency" among many students who are unable to seek assistance for their queries instantly. This online learning and teaching module has been designed precisely to fill that gap and bring together human knowledge and AI through the medium of conventional course work. The reason for choosing this topic for research is the pressing need to create an interactive yet cost effective and efficient learning environment for learners. Through the proposed AI tutor for Retrieval Augmented Generation (RAG), we intend to examine how local machine learning can enhance students' learning experience.

1.3 Methodology

In order to make sure that the technical accuracy and applicability of the EduBridge platform is achieved, two different kinds of sources for data have been used. The process adopted involves a systematic method for data collection starting from needs identification of users and ending with software architecture development. The use of both observation and technical literature has helped in balancing both theoretical and practical approaches during the process. This multidimensional technique for conducting research has ensured that the result is something that not only satisfies university requirements but also helps tackle real world problems in the domain of online education.

1.3.1 Primary Sources

The main sources of data collection involved direct interactions as well as conducting research, including:

- Direct interaction with research supervisors in university to ensure that the architecture design of the platform is academically sound.
- Conducting tests to measure the level of accuracy in responses from the AI tutor, using actual lesson transcripts.
- Development and debugging of the Laravel backend and MySQL database designs.
- Collection of feedback from peers while demonstrating the user interface and payment processes.

1.3.2 Secondary Sources

The secondary data served as the theoretical basis and technical guidelines for the implementation of the project through the following ways:

- The study of the documentation on the API usage in the Gemini 1.5 Flash and 2.0 Flash versions.
- Review of the recommendations and security measures related to money transactions in the Laravel framework.
- The review of other learning management systems and identification of their deficiencies in relation to user experience.
- Review of the scientific articles related to the performance of RAGs in education.

1.4 Objectives

The aims of the system have been categorized into general and specific ones in order to achieve a balance between the vision and the implementation aspects. The objectives form a guide to developing the application in a way that all implemented features have a purpose towards improving the user's experience.

1.4.1 Broad Objective

The overall goal in this case will be to come up with a system that enables effective transfer of knowledge between the instructor and the learner in a very secure and interactive manner. It will be necessary to create an environment that will be sustainable in nature, which ensures that learning takes place in a personal and cost-effective manner.

1.4.2 Specific Objectives

- Implement a context aware AI Tutor using Retrieval Augmented Generation to provide students with instant, lesson specific support.
- Develop a secure multi gate payment system integrating Stripe and SSLCommerz for automated and reliable financial transactions.
- Create a robust instructor dashboard for content authoring, transcript management, and revenue tracking with a defined 70/30 split logic.
- Design an intuitive, responsive user interface that supports high fidelity video streaming and dynamic course progress tracking.
- Establish a centralized administration module for user moderation, course quality assurance, and financial auditing.

1.5 Process Model

The implementation process involved the use of the Agile Scrum Process model. This iterative process was preferred due to the fact that it is highly flexible and adaptive in nature.

Some of the reasons why the Agile Scrum Process model was chosen are:

- The model helps in dealing with complexities associated with integrating third-party vendors through sprints lasting two weeks each.
- It aids in quickly designing and testing out the logic behind the AI tutor in order to iron out any bugs.
- Provides visibility of the development process on a day-to-day basis.
- Enables incremental delivery of functionalities where basic learning management functionalities can be completed while advanced AI functionalities are in refinement.

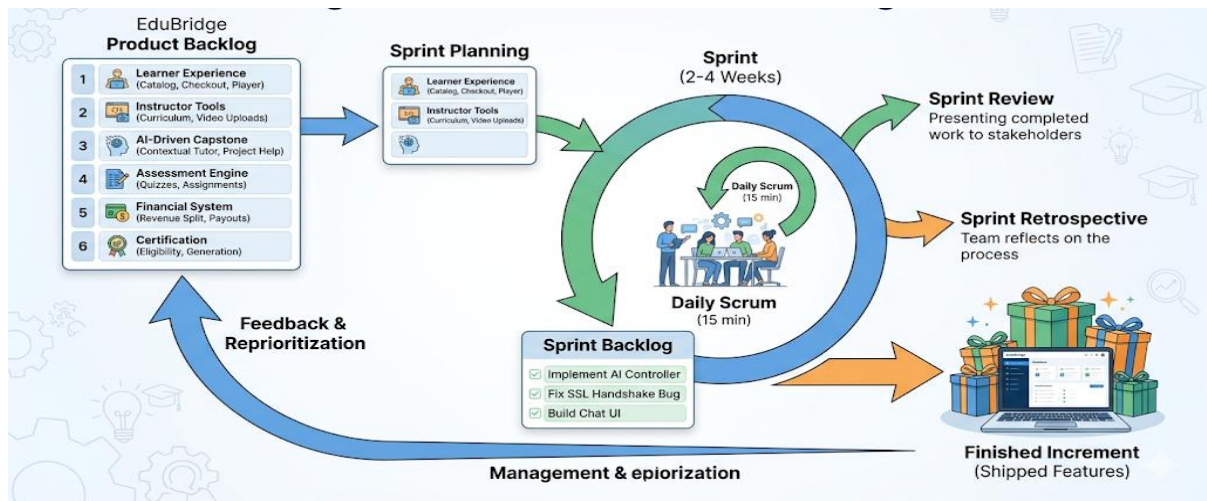


Figure 1.1: The Agile Scrum Process Model for EduBridge Development.

1.6 Feasibility Study

A detailed feasibility study was conducted in order to evaluate the viability of the project on technical, financial, and organizational levels. This way it became possible to foresee potential problems and identify required resources before the full-scale implementation process. Considering the analysis made of all three above-mentioned aspects, the likelihood of the project's successful completion was considerably increased.

1.6.1 Technical Feasibility

From a technical perspective, the project seems feasible since it will involve using well-known technologies that have been recently developed. The Laravel framework will provide a reliable and efficient environment for developing the backend, whereas the Gemini API will allow for more sophisticated natural language processing. The hardware requirements of the project are typical for contemporary Web applications, and cloud-based video storage services such as Amazon S3 will make it unnecessary to overload local server.

1.6.2 Economic Feasibility

This online learning and teaching system is economically viable due to its affordable starting point and scalability in earning. The use of open source software and pay as you go APIs helped

keep the development expenses low. The 70:30 split revenue model between teachers and the learning platform guarantees economic viability in the long term. This means that the learning system will be able to earn money to pay for server and maintenance expenses as more users join.

1.6.3 Operational Feasibility

The system is highly operational since it has been built with the end users in mind. The interface of the AI Tutoring System is user-friendly for learners and teachers. In addition, the automatic nature of the AI Tutor and the payment systems implies that fewer personnel are required to run the whole system.

1.7 Structure of the Report

In order to present a logical sequence of the life cycle of the project, the report is organized in ten detailed chapters:

- Chapter 1: Introduction - Presents information about the background of the project, its key objectives, agile approach used in the project, as well as the feasibility of the online learning and teaching platform.
- Chapter 2: Organizational Overview – Presents relevant information about the environment of the practicum, the development team structure, and the organizational context for the project.
- Chapter 3: Requirement Engineering – Presents functional and non-functional requirements that define the core characteristics of the system.
- Chapter 4: Software Analysis – Presents the architecture and behavior of the system using detailed UML Use Case and Class diagrams.
- Chapter 5: Project Management – Presents task scheduling, resources management, and timeline tracking used for the maintenance of the agile sprints in the development process.

- Chapter 6: Project Planning and Estimation – Presents strict software estimation based on mathematics, especially Function Point Analysis (FPA).
- Chapter 7: System Design- Highlights the detailed technical design of the system, which includes Data Flow Diagram (DFD), database normalization up to Third Normal Form (3NF), Entity Relationship Diagram (ERD), and front-end UI prototype.
- Chapter 8: System Quality and Testing- Provides information regarding the process of quality assurance, methodology of testing, and the test case used to ensure system efficiency.
- Chapter 9: Ethical Considerations- Highlights the responsibilities that the software holds from the ethical perspective, such as data privacy, intellectual property, AI bias, and economic sustainability of the software.
- Chapter 10: Conclusion- Provides an overview of the successes attained in the course of the practicum, and assesses the practical significance of the practicum, its limitations and recommendations for the future.

Chapter 2. Organizational Overview

2.1 Organizational Overview

This chapter will describe SourceExpert in detail, the company in which this practicum took place. The chapter will highlight the core values of the organization, including its vision and mission, and describe the various technical services provided by the organization. Moreover, it will discuss the organizational structure inside the firm as well as the job designation during the internship period.

2.2 Organization Vision

The vision statement for SourceExpert is to be the first global technology partner for startups and enterprises, known for offering excellent quality and innovations while maintaining a client first approach throughout. The company aspires towards creating an environment whereby every entrepreneur in the world has the opportunity to have his/her ideas implemented by world-class engineers.

2.3 Organization Mission

The mission of SourceExpert is to provide the best MVP solutions to small, medium and large-scale business entrepreneurs across the globe. The company's mission is to connect innovative ideas with engineering skills and a global outlook to help create market-ready solutions to help transform visions into reality.

2.4 Organization Services

SourceExpert provides an extensive array of software services that are production-ready through the collaboration of its certified personnel in the United States and Bangladesh. This encompasses:

- Web Development: Creation of scalable and high-performance web applications using up-to-date technologies.
- Mobile Applications: Construction of native and cross-platform mobile apps for varied business purposes.

- Artificial Intelligence/Machine Learning Solutions: Application of sophisticated artificial intelligence and machine learning algorithms to address intricate data challenges.
- Cloud Services: Optimization of cloud architectures and strategies for effective performance.
- User Interface/User Experience Design: Creation of user-friendly interfaces and experience designs.
- Staff Augmentation: Provision of specialized engineers to assist in external development processes.

In addition to these core services, SourceExpert places particular emphasis on mentoring engineers and collaborating with them. During my period at SourceExpert, I made contributions to the company's knowledge repository by recording guidelines for integrating APIs. I also took part in coding reviews which allowed for the reuse of modular codes that were developed for specific projects in other company projects.

2.5 Organizational Structure

SourceExpert's organizational structure is aimed at promoting teamwork and technical expertise within its worldwide branches. The structure of the organization is shown through the organogram in Figure 2.1 and represents a very efficient three-tier structure that acts as a link between the executive layer of the organization and its operational departments.

The first tier of the organization includes the Chief Executive Officer, who has an overarching responsibility for overseeing the overall strategic direction of SourceExpert. Under the leadership of the CEO, the second tier of the organization consists of the tactical layer, which is represented by three major pillars and includes the Head of Engineering, the Product Director, and the Operations Manager. The third tier of the organization, or the operational execution tier, is represented by:

- Engineering Department: Led by the Head of Engineering, this department implements the technical vision and oversees software quality via the Web

Development Team, the AI/ML Department, and the Quality Assurance & Testing Department.

- **Product Department:**Led by the Product Director, this department focuses on ensuring that the software satisfies market and customer demands by managing the UI/UX Design Department and Product Management.
- **Operations Department:**Directed by the Operations Manager, this department is responsible for internal and external coordination through Human Resources & Administration and Customer Relationship Management.

This is an organized process that ensures smooth communication among the developers, designers, and project managers. It ensures that complex AI-powered solutions such as EduBridge are created on schedule and within budget limits.

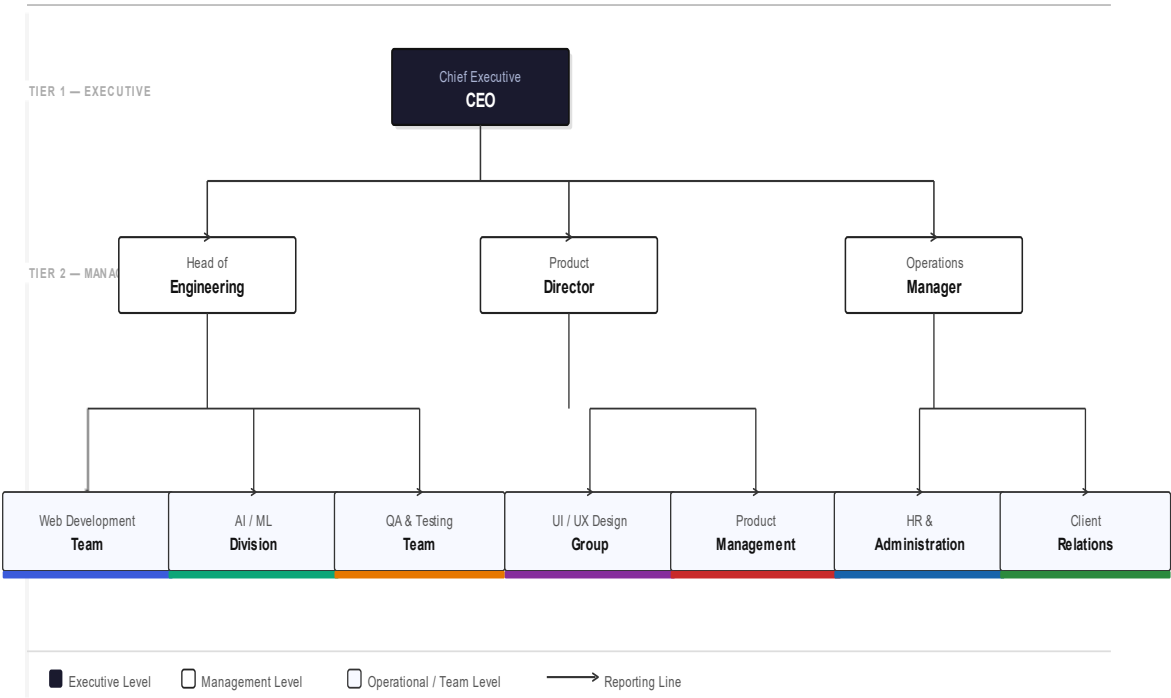


Figure 2.1: Organizational Structure of SourceExpert.

2.6 My Position in this Organization

My assigned role during the practicum was Intern – Software & Application Development (ID: 2026-003), which started from February 1, 2026 at our Bangladesh office in Uttara,

Dhaka. The main tasks were related to the full stack development of the EduBridge product, which entailed applying engineering skills for bridging the gap between the conceptual idea and the production-ready learning management system. In particular, my responsibilities included:

- Architectural Design – Development of the core logic of the AI-driven contextual assistant based on Laravel and Gemini API.
- System Integration – Implementation of secure payment flows in order to manage the automated revenue split through SSLCommerz and Stripe.
- Quality Engineering – Performance of thorough unit testing and debugging of the product in order to solve the problems with SSL handshake and API versioning.
- Collaborative Development – Synchronization with the design team in order to maintain the responsiveness of the user interface according to the needs of the instructor and learner.

As an intern in the Software & Application Development team, I had the chance to witness how professional engineering teams develop digital products on a large scale.

2.7 Address of the Organization

Company Name: SourceExpert

Company Address: Rose Garden, House 36, Road 13, Sector 11, Uttara Model Town, Dhaka 1230

Chapter 3. Requirement Engineering

3.1 Requirement Engineering

Requirement Engineering (RE) refers to the process of eliciting, documenting and managing the requirements of a software system. It guarantees that the software being developed meets the requirements of the stakeholders (instructors, students and platform administrators). Requirement engineering forms a link between the real-world requirements of users and the technical requirements of developers. Through the process of agreeing to the requirements of the system, it reduces chances of failure of the project, and ensures that the end product – in this case the EduBridge platform, has value. The main three steps of requirement engineering include:

- I. Requirement Elicitation: Eliciting requirements of the stakeholders through interviews, surveys, meetings, observation and research on current available solutions.
- II. Requirement Analysis: Analyzing and prioritizing requirements, identifying conflicts, and inconsistencies and checking the technical feasibility of the requirements.
- III. Requirement Specification: Documenting requirements in a specified format mostly in the software requirement specification document (SRS).

Detailed descriptions of the above steps are provided below with respect to the EduBridge: A Comprehensive E Learning Platform project.

3.2 Requirement Elicitation

Requirement Elicitation is a process of collecting data from stakeholders like educators, learners, and organizational management regarding the required attributes and features of the software. It is an essential phase because it allows to determine hidden needs of users such as how educators would want to control the 70 percent revenue share of theirs. The process of collecting the information is done by asking questions, making interviews or surveys, and observing learning processes in order to identify problems and expectations. The aim is to collect the clear and complete information about the real needs of users since we have to create the system that will be able to cope with competition.

3.2.1 Requirement Gathering

Direct interviews and surveys of stakeholders who include educators willing to work online and learners willing to use the convenient platform of learning were made in order to collect the data regarding the needs, expectations, and functional requirements of the system. Also, there was made a comparative analysis of existing Learning Management Systems (LMS) and the gaps in the provision of support for learners. That allowed integrating the AI Tutor feature into this project.

3.3 Requirement Analysis

At this stage, requirements will be reviewed to determine unique features based on user stories and technical requirements. In this stage, it is important to classify the requirements as functional or non-functional and ensure that each feature matches the goals of the project. Filtering of the requirements is done based on the technical stack of the project—Laravel and Gemini AI API—to ensure that all the features are buildable and functional. The key requirements as determined by this thorough analysis are listed below:

1. Instructor Requirements

- Apply to become an instructor through a dedicated application process.
- Access a specialized dashboard with course analytics and earnings reports.
- Create, edit, unpublish, and delete courses.
- Utilize a dynamic Course Builder to efficiently manage modules and lessons.
- Build interactive quizzes with multiple choice questions and create practical assignments.
- Grade student assignment submissions and monitor assessment dashboards.
- View the enrolled students' roster and revoke student access if necessary.
- Submit finalized courses for Admin approval and preview them before publication.
- Post announcements to communicate directly with enrolled students.
- Read student reviews and post direct replies.

- Request financial payouts from accumulated course earnings based on the 70/30 split.
- Manage personalized settings, profile information, password, and notification preferences.

2. Learner Requirements

- Browse the public course catalog, view instructor profiles, and read public reviews.
- Apply to become an instructor via the "Teach with Us" portal.
- Join a waitlist for courses that are currently unavailable.
- Checkout and enroll securely using multiple payment gateways (Stripe and SSLCommerz).
- Access a dedicated Learning Environment (Course Viewer) to consume course content.
- Mark lessons as complete to track individual learning progress.
- View instructor announcements directly within the course context.
- Attempt quizzes and submit assignments for instructor grading.
- Leave ratings and reviews for completed courses.
- Claim, preview, and download certificates upon course completion.
- Interact with a smart AI Tutor feature to ask questions and get intelligent course related assistance instantly.
- Manage personal profile, account settings, and password securely.

3. Admin Requirements

- Access a high-level admin dashboard to oversee platform operations.
- Manage all users: manually create, promote to instructor, suspend, or delete user accounts.

- Review, approve, or reject new instructor applications.
- Review, approve, or reject new course submissions from instructors.
- Manage global course categories with full CRUD (Create, Read, Update, Delete) capabilities.
- Oversee certificates globally and toggle their public validity.
- Moderate course reviews by toggling their visibility or deleting inappropriate content.
- Monitor platform revenue analytics, financial health, and AI Tutor API usage.
- Approve, reject, or simulate payment transactions.
- Review instructor payout requests and systematically mark them as paid after validating the 70/30 revenue share calculation.
- Broadcast global announcements to targeted user roles (Learners, Instructors, or All) to facilitate platform-wide communication.

Other requirements that will be met by the clients include:

- a. A user-friendly environment with a good-looking UI.
- b. Specialized dashboards built especially for the Admin, Instructor, and Learner roles.
- c. Login and Registration with strict data validation.
- d. A real-time global notification system available to everyone.
- e. Public Verification System where any user can verify the validity of their downloaded certificates through a unique code.

3.4 Requirement Specifications

In this step, detailed requirements for the system are documented. Both functional and non-functional requirements are included here.

3.4.1 User Requirements

- Instructor can register on the platform and submit an instructor application.
- Instructor can access a comprehensive course builder after admin approval.
- Instructor can add, edit, and delete courses, modules, and lessons.
- Instructor can create and manage assessments (quizzes and assignments).
- Instructor can evaluate learner submissions and provide grades.
- Instructor can view earnings based on the revenue split and request payouts.
- Administrator can approve or reject instructor applications and manage user roles.
- Administrator can manage course categories and platform settings.
- Administrator can review financial transactions and process instructor payouts.
- Learner can browse published courses in the public catalog.
- Learner can purchase and enroll in courses securely.
- Learner can access learning materials, submit assignments, and take quizzes.
- Learner can track their progress and earn certificates.
- Learner can chat with the AI Tutor for personalized assistance.
- Learner can create and manage a personal account securely.

3.4.2 System Requirements

1. User Registration and Profile Management

- User clicks the "Sign Up" button to go to the registration page.
- User provides name, email, phone number, and password for registration.
- After successful registration, the user is redirected to their respective dashboard.
- User clicks the "Profile Settings" option to update personal information and manage security.

2. Instructor Application and Approval

- A registered user navigates to the "Teach with Us" page.

- User submits details regarding their expertise and qualifications.
- Admin reviews the application in the Admin Dashboard under "Pending Applications."
- Admin clicks "Approve" to grant the user instructor privileges or "Reject" with a reason.

3. Course Creation and Management

- Instructor clicks the "Create New Course" button in the Instructor Dashboard.
- Instructor inputs course details (title, description, price, category, cover image).
- Instructor uses the Course Builder to add modules and lessons.
- Instructor clicks "Submit" to send the course for admin review.
- Admin reviews the course in "Course Approvals" and clicks "Approve" to make it public.

4. Assessment Management (Quizzes and Assignments)

- Inside a course module, Instructor clicks "Add Quiz" or "Add Assignment."
- For quizzes, Instructor adds multiple choice questions and defines correct options.
- For assignments, Instructor sets instructions, deadlines, and total marks.
- Instructor saves the assessment for learners to access.

5. Course Browsing, Waitlist, and Enrollment

- Learner visits the public course catalog.
- If a course is unavailable, Learner clicks "Join Waitlist".
- If available, Learner clicks "Enroll Now" and proceeds to checkout.
- Learner selects preferred payment gateway (Stripe or SSLCommerz).
- Learner completes payment on the gateway portal and is redirected back.
- Upon success, the course is added to the "My Courses" section.

6. Learning, AI Tutor, and Evaluation

- Learner opens an enrolled course and accesses lesson content in the Course Viewer.
- Learner marks lessons as "Complete" to update progress bars.

- Learner uses the "Ask AI" interface to submit course related questions and receive automated, intelligent answers instantly.
- Learner takes quizzes and receives immediate automated feedback.
- Learner submits assignments by uploading files or text.
- Instructor receives notifications for submissions, reviews them, and assigns grades.

7. Certification and Public Verification

- Upon 100% course completion, Learner clicks "Claim Certificate".
- Learner previews and downloads the generated PDF certificate.
- Anyone can visit the "/verify-certificate" route, input the certificate code, and view its validity status.

8. Payout Management

- Instructor views accumulated earnings in the Dashboard (calculated at 70% of total course sales).
- Instructor submits a Payout Request for a specific amount.
- Admin reviews the payout request and clicks "Mark as Paid".
- Instructor's balance is updated upon successful payout.

9. Review Moderation

- Admin navigates to the "Reviews" section in the admin panel.
- Admin views a list of all course reviews.
- Admin clicks "Toggle Visibility" to hide an inappropriate review, or "Delete" to remove it entirely.

10. Global Announcement and Notification Flow

- Admin clicks the "Global Announcements" button in the Sidebar.
- Admin inputs the Announcement Title, selects the Target Audience from a dropdown, and writes the message.

- Upon clicking "Push Global Announcement," the system dispatches in-app notifications to all users in the selected category.
- Users receive a real-time visual alert on their respective dashboards.

3.4.3 Functional Requirements

Functional requirements describe what a system should do. They explain the features that users can use. Below are the functional requirements of EduBridge.

1. Authentication and Authorization

- The system shall allow users to register as learners by default.
- The system shall authenticate users securely and route them to their respective dashboards based on roles (Admin, Instructor, Learner).
- The system shall provide password visibility toggles on login/registration forms.

2. Course Management (Instructor/Admin)

- The system shall allow instructors to create, read, update, unpublish, and delete (CRUD) courses.
- The system shall allow instructors to submit courses for an admin review cycle before publication.
- The system shall allow admins to manage global course categories.
- The system shall allow instructors to view the enrolled students roster and revoke access if necessary.

3. Assessment System (Instructor/Learner)

- The system shall enable instructors to create quizzes with various question types.
- The system shall allow instructors to define assignments with grading rubrics.
- The system shall allow learners to attempt quizzes and record their scores automatically.
- The system shall allow learners to submit assignment files and instructors to input grades.

4. Enrollment and Payment (Learner/Admin)

- The system shall display a public catalog of published courses.
- The system shall allow learners to join and leave course waitlists.
- The system shall support both international (Stripe) and local (SSLCommerz) payment gateways for checkout.
- The system shall grant learners access to course materials immediately after successful payment webhook confirmation.
- The system shall allow administrators to manually simulate payment successes for testing or manual overrides.

5. Learning Tools and AI Integration

- The system shall track the completion status of individual lessons for enrolled learners.
- The system shall provide a smart AI tutor interface for learners to ask course related questions dynamically.
- The system shall allow instructors to post announcements directly to course learners.

6. Certification System

- The system shall generate and issue a downloadable certificate upon 100% course completion.
- The system shall enable anyone to verify the certificate through a unique verification code.
- The system shall enable admins to turn the validity of the certificate ON/OFF.

7. Finances Management & Revenues Split

The system shall log all transactions and enrollments.

- The system shall compute revenues on a strict 70/30 revenues split (Instructor 70%, Platform Admin 30%) per each successful course purchase.
- The system shall enable instructors to apply for payments from their 70% and admins to mark them as paid.

3.4.4 Non-Functional Requirements

Non-functional requirements are descriptions of how a system is supposed to behave. These requirements concern issues like performance, security, reliability, and usability.

1. Security

- The system shall enforce data integrity using strict validation rules.
- Passwords shall be securely hashed using algorithms like Bcrypt.
- Middleware for RBAC should be implemented on the platform to secure access to critical instructor and admin routes.
- All transactions should be done through the payment gateways (Stripe/SSLCommerz) securely, without storing any raw data of credit cards on the platform.

2. Usability

- The platform shall implement responsive design with seamless performance on desktop, tablet, and mobile devices using Tailwind CSS.
- The forms should have live frontend validations to avoid user mistakes during data entry.
- Dashboards should be easy and unique for different roles (Admins, Instructors, and Learners) to avoid any navigation mistakes.
- The Blade reusable components should be used throughout the application to have consistent UI.

3. Scalability and Performance

- The system shall have the capability to store more users, courses, and transactions with its relational database structure (MySQL).
- The AI Tutor API calls should not block application threads and should be processed asynchronously.
- Vite should be used by the application for optimized frontend assets bundles.

4. Reliability

- The system should be capable of handling webhooks payments confirmations asynchronously to avoid missing enrollments due to the slow network.
- Database error logs and invalid payments attempts should be logged by the system to help administrators troubleshoot issues.

5. Maintainability

- The system will be developed in the Laravel MVC framework, guaranteeing modularity and maintainable codes.
- The system's code will use different controllers for the learner, instructor, and admin moderation functionalities.

3.5 Use Case Diagram of the System

Use Case Diagram is a behavioral model used for identifying the key actors and the respective functionalities of the actors in a particular software environment. This diagram shows the boundary of the system, showing how the external entities make use of the internal functionalities of the software for accomplishing certain purposes. This diagram is crucial for determining the boundary of the project and making sure that the functional requirements are considered during the development and testing of the system.

3.5.1 Functional Description of Actors

The actors within the EduBridge system include the following:

- **Learner (Student):** Main user who navigates through the catalogues, manages his/her profile, and enrolls in courses.
- **Instructor (Teacher):** The content creator who manages the full course lifecycle, including authoring content, building assessments, and tracking financial payouts.
- **Admin (System Controller):** The system moderator responsible for user moderation, approving instructor applications, auditing finances, and managing infrastructure.
- **External Systems:** Third-party integrations that support core functions, including Payment Gateways (Stripe/SSLCommerz), the AI Engine (Groq API), and media storage (Amazon S3 / Local).



Figure 3.1: Use Case Diagram for the EduBridge Platform

Chapter 4. Analysis

4.1 Software Analysis Pattern

The EduBridge software uses the Model-View-Controller (MVC) software analysis and architecture design. This is because this model effectively segregates the logic of the application from its UI making the application scalable, maintainable and organized in code architecture which is required in an educational application.

- **Model:** Consists of the database structure and logic. In this case, the Object-Relational Mapping (ORM) takes care of complex database relationships like mapping Users to Enrollments, Courses to Modules, and AI Transcripts to Lessons ensuring database consistency in the MySQL database.
- **View:** Consists of the presentation layer. In this case, it includes the User Interface where the application shows dynamic data like interactive video lessons, AI Tutor chat interface, and financial dashboard for both learners and instructors.
- **Controller:** Serves as the central processing unit between the view and the model. It receives all user input actions (e.g. when submitting an assignment, requesting a payout or calling the Groq API) and then applies the relevant business logic, validations and security rules and updates the model or the view accordingly.

Furthermore, the system uses the service-oriented architecture (SOA) design model for its external boundaries using secure Webhooks for interaction with external third-party payment gateways (Stripe/SSLCommerz) and asynchronous REST API calls for the Groq-based RAG AI engine.

4.2 Activity Diagram

Activity diagrams model the dynamic behaviors of the system, showing the transitions of control from one activity to another. As far as EduBridge is concerned, these diagrams show how the system behaves when interacting with users and how data is handled in the database. Decision points are included to make sure that errors, limitations, and security measures are taken into consideration.

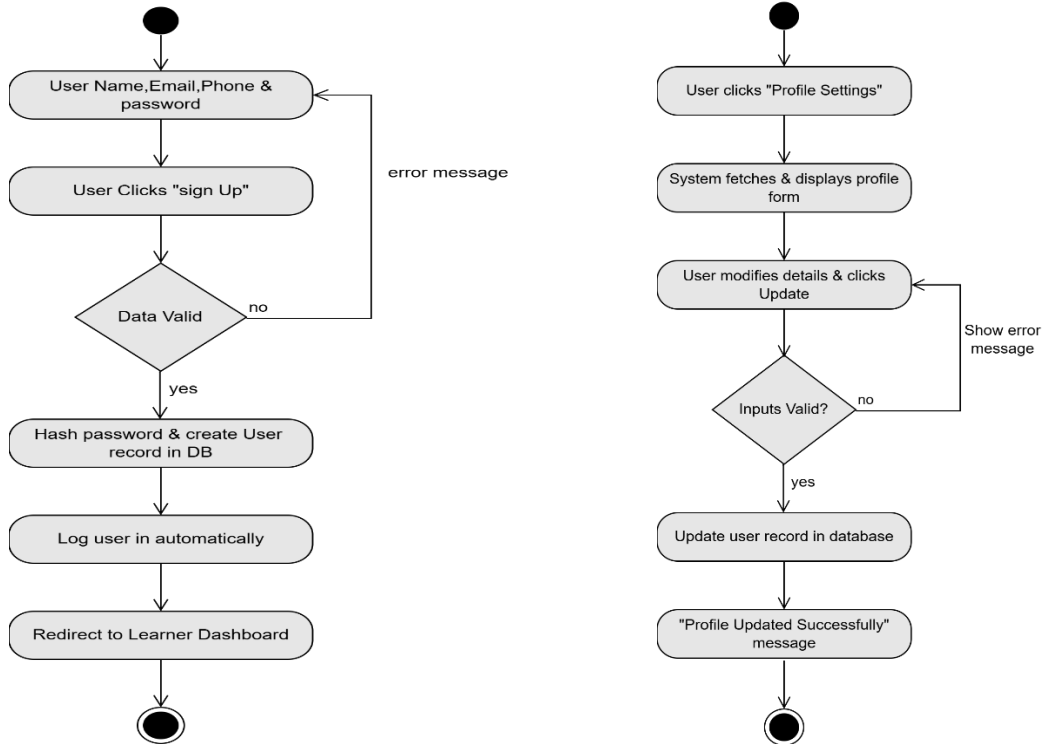


Figure 4.1: Activity Diagram for User Registration & View User Profile

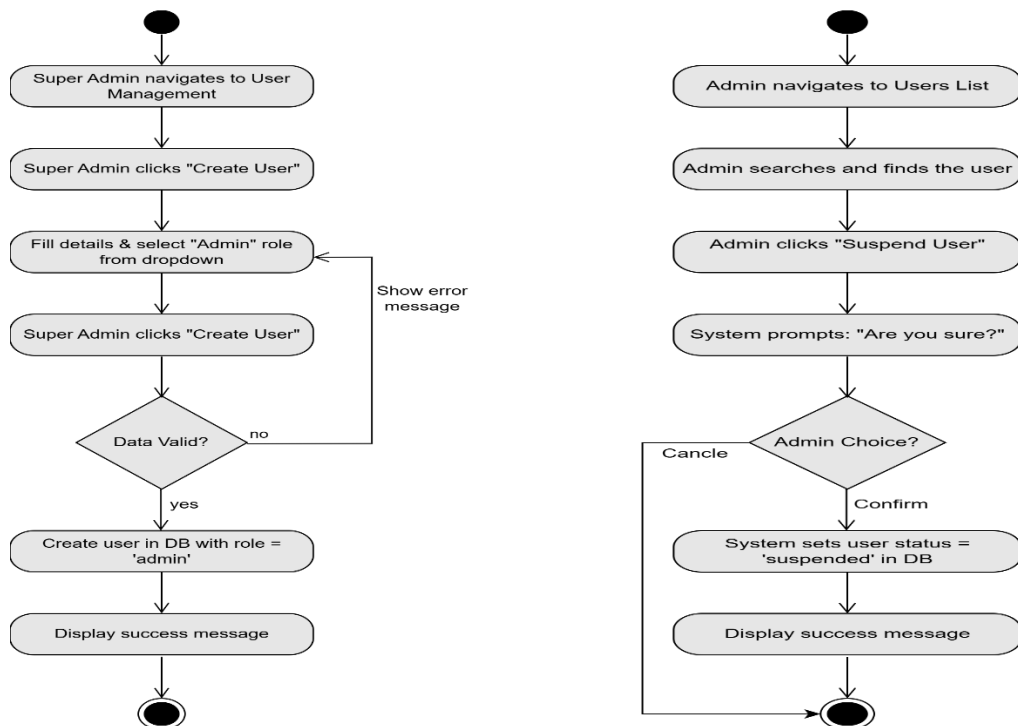


Figure 4.2: Activity Diagram for Add Admin & Manage Customer

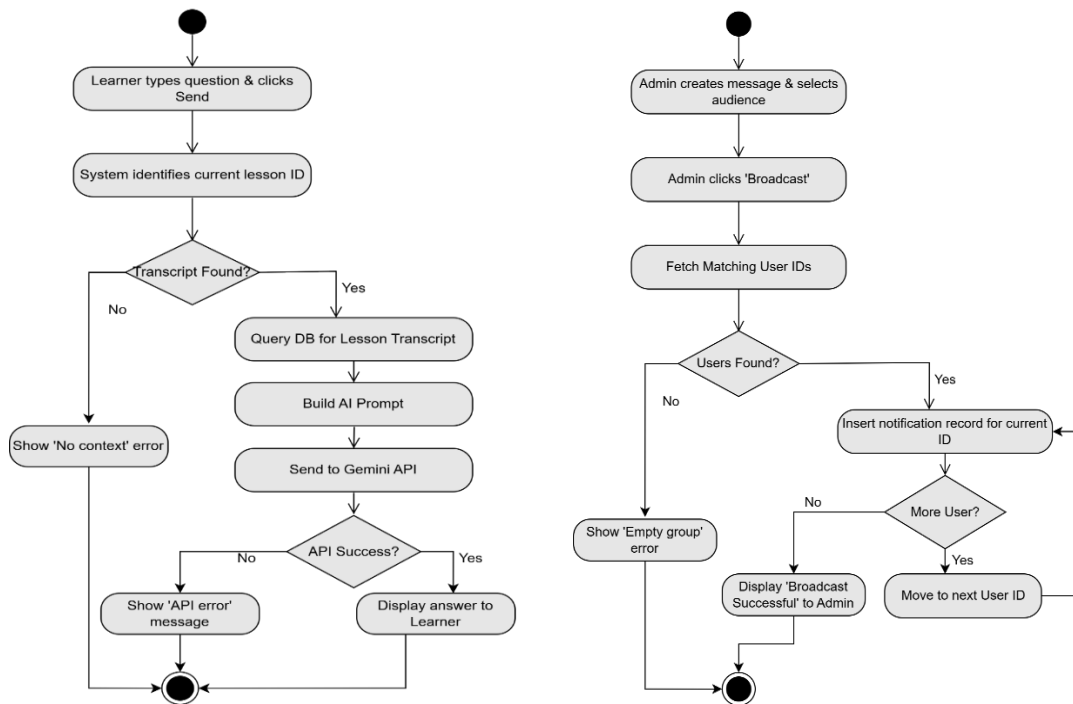


Figure 4.3: Activity Diagram for AI Tutor Interaction & Global Announcement

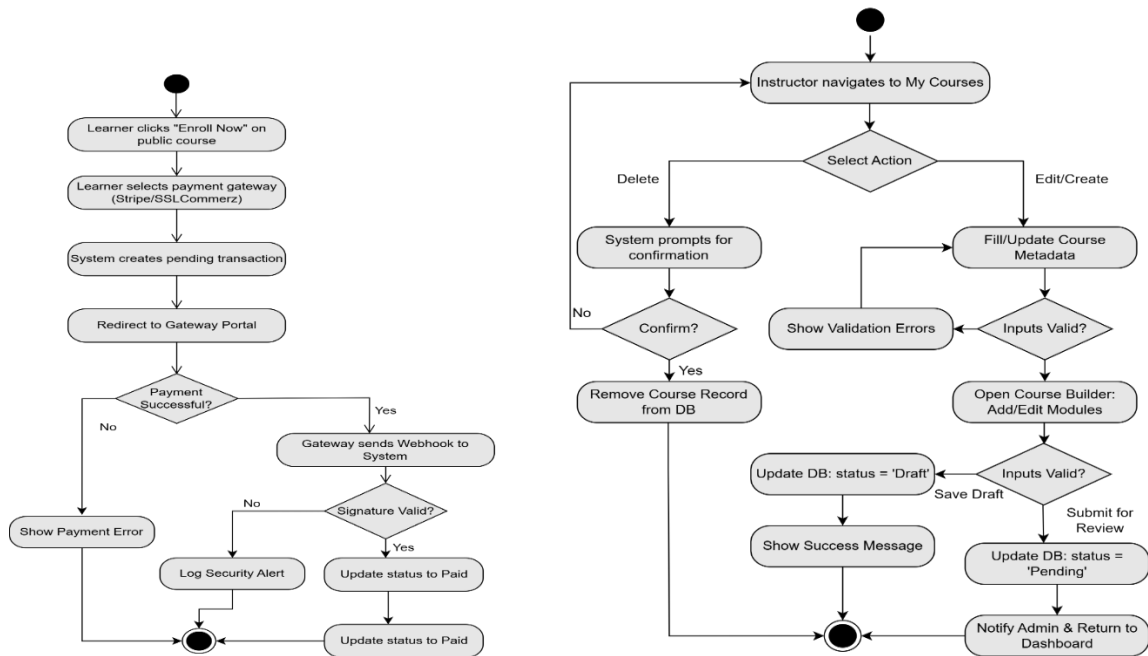


Figure 4.4: Activity Diagram for Course Enrollment with Payment & Course Creation

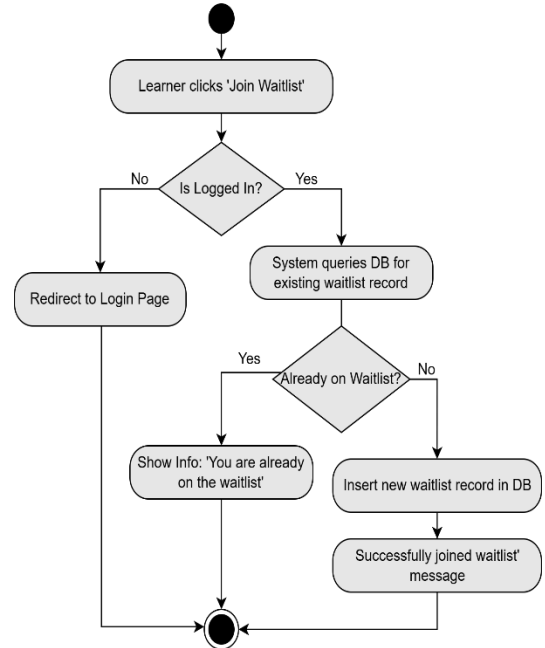
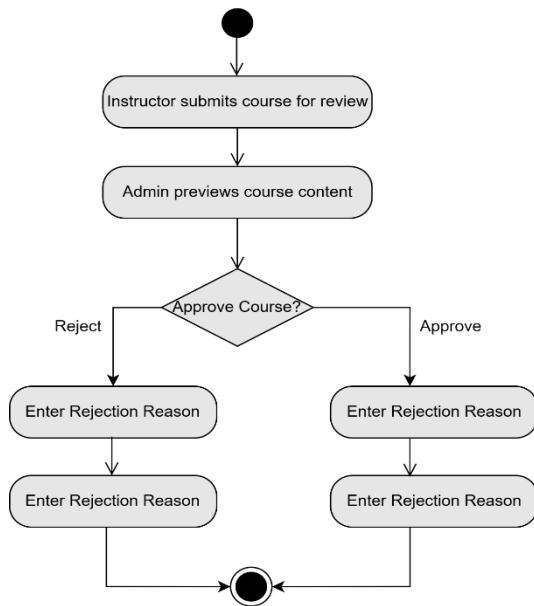


Figure 4.5: Activity Diagram for Course Approval& Waitlist

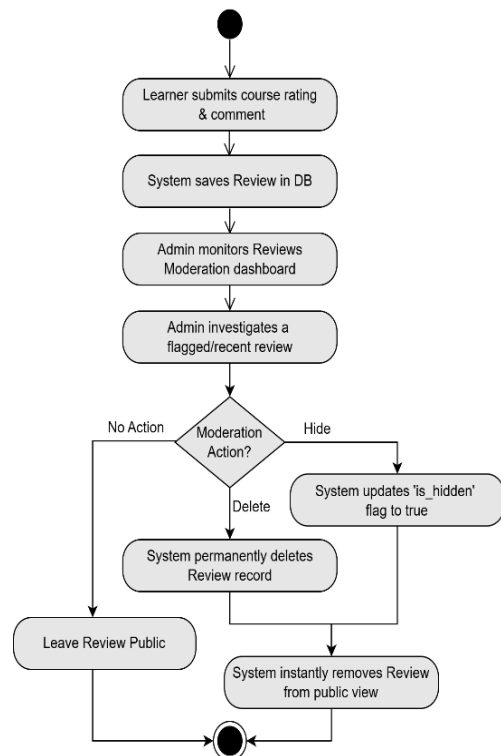
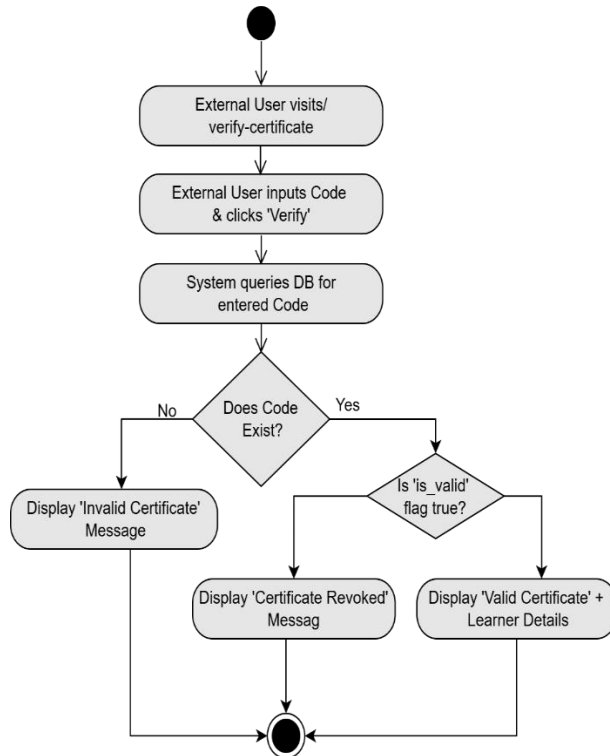


Figure 4.6: Activity Diagram for Certificate & Review

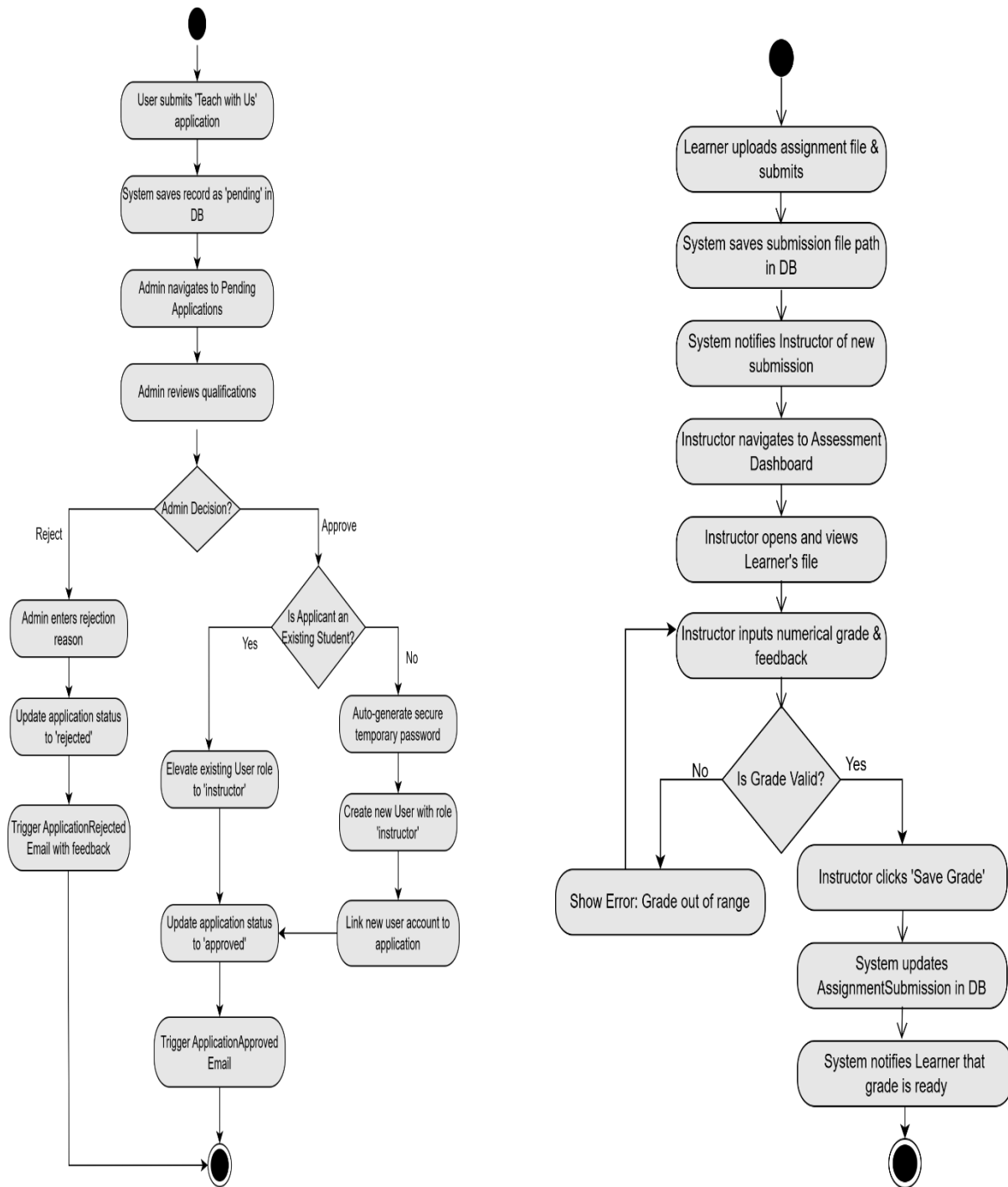


Figure 4.7: Activity Diagram for Instructor Application & Instructor Grading Assignments

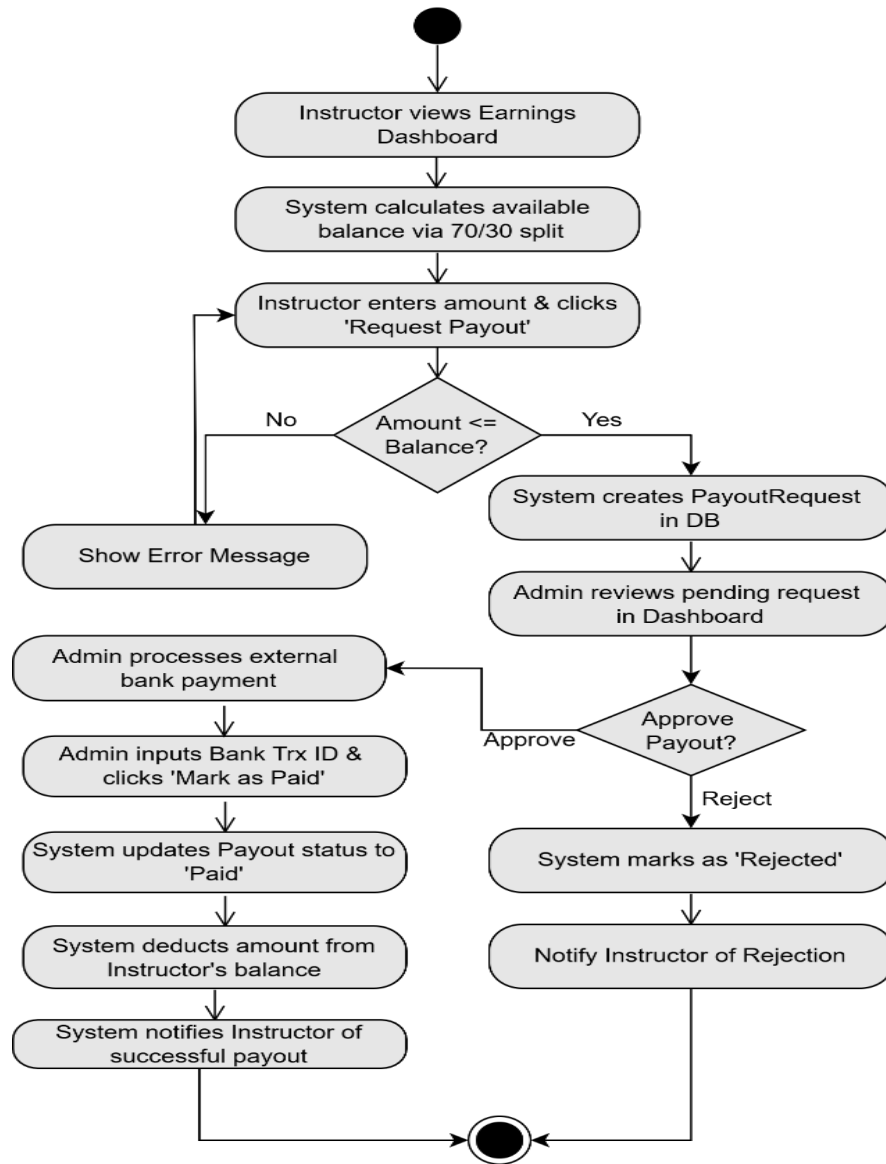


Figure 4.8: Activity Diagram for Payout Request

4.3 Swim-lane Diagram

Whereas traditional activity diagrams only represent the sequence of events, swim lane diagrams help split these events and clearly specify who or what is supposed to carry out these activities. The responsibility of tasks in the EduBridge system is divided among the following functional swim lanes: Human actors (Learner, Instructor, Admin); System (Backend Controller); Storage (Database); and External APIs.

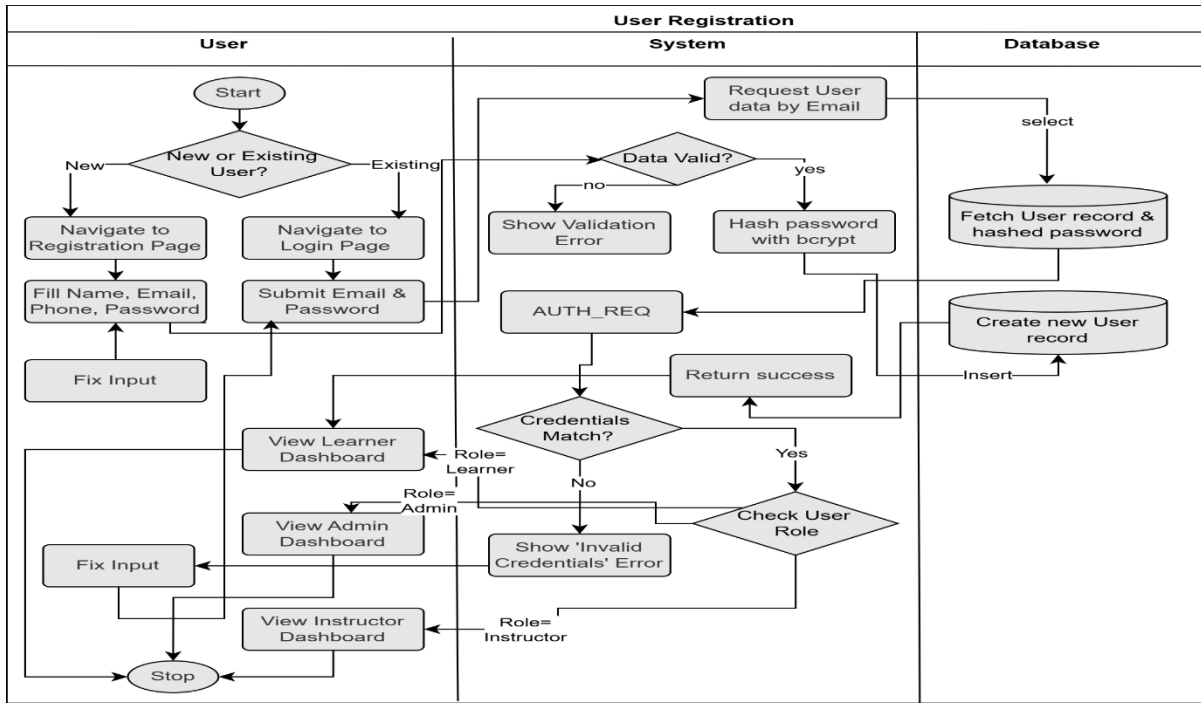


Figure 4.9: Swim-lane Diagram for Authentication & Registration

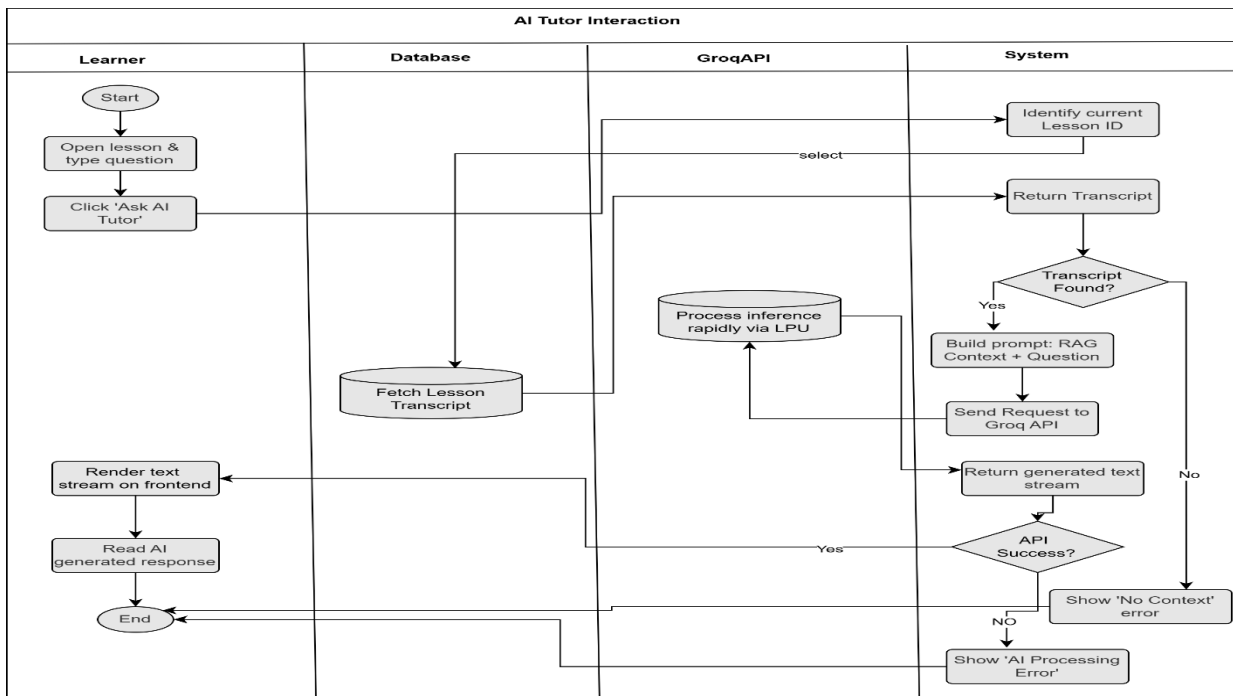


Figure 4.10: Swim-lane Diagram for AI Tutor Interaction

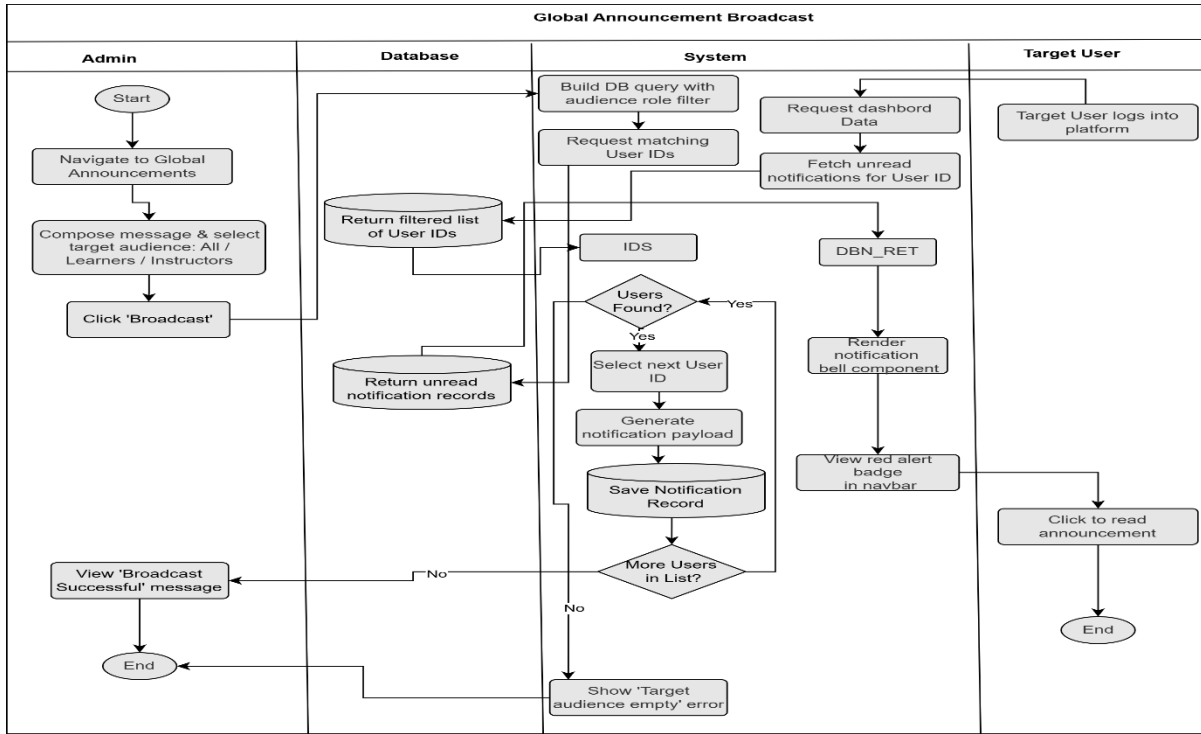


Figure 4.11: Swim-lane Diagram for Global Broadcast

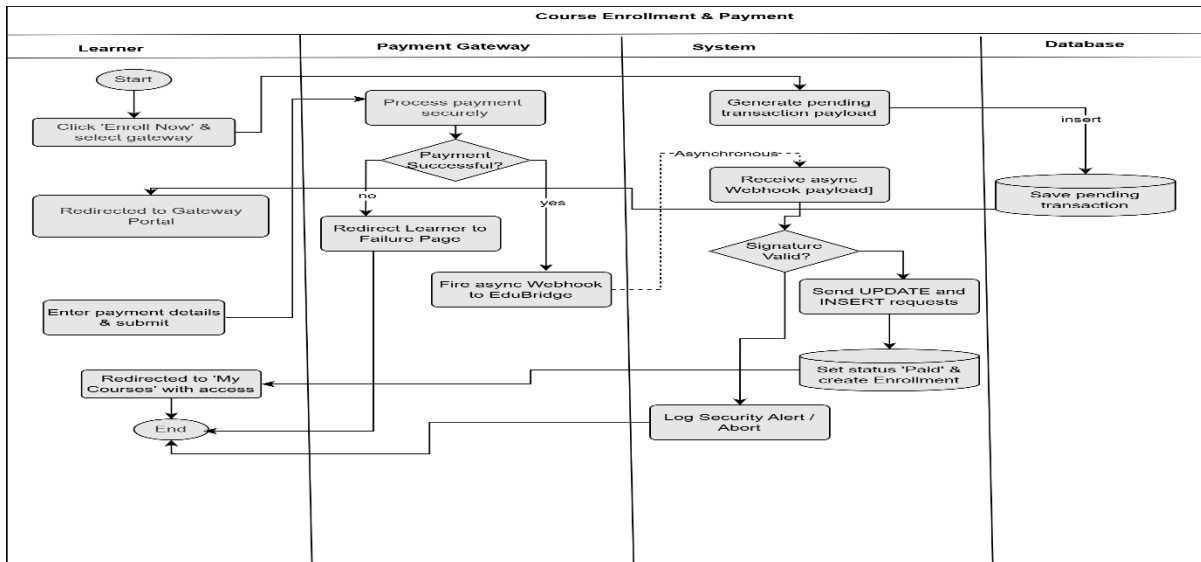


Figure 4.12: Swim-lane Diagram for Course Enrollment

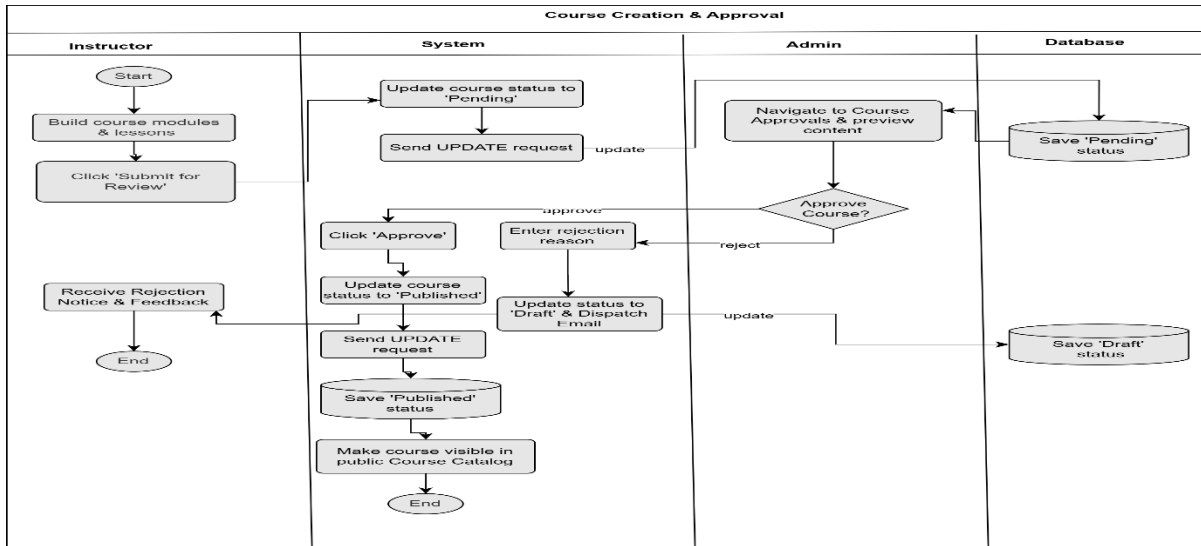


Figure 4.13: Swim-lane Diagram for Course Creation & Approval

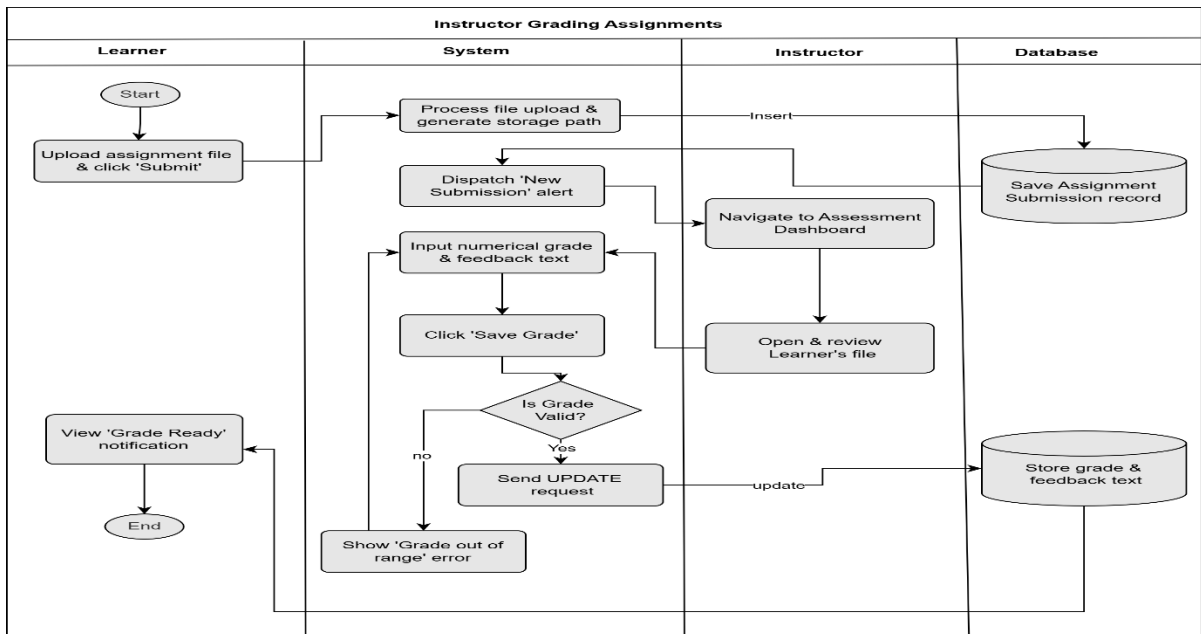


Figure 4.14: Swim-lane Diagram for Instructor Grading Assignments

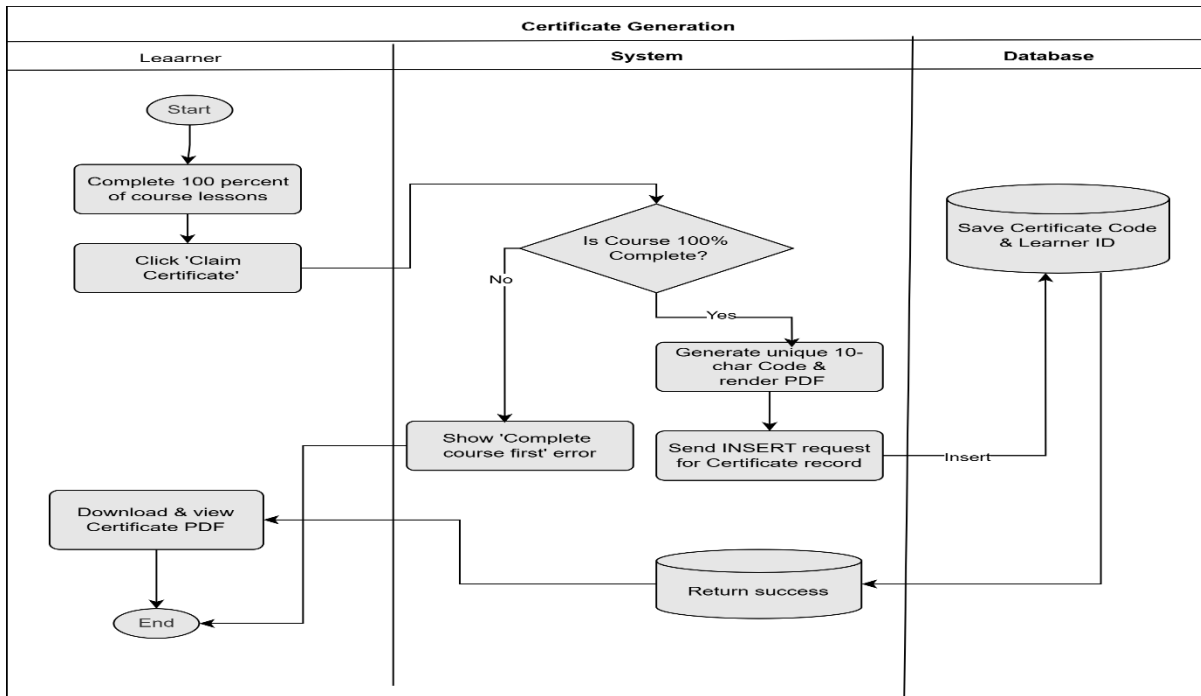


Figure 4.15: Swim-lane Diagram for Certificate Generation & Claiming

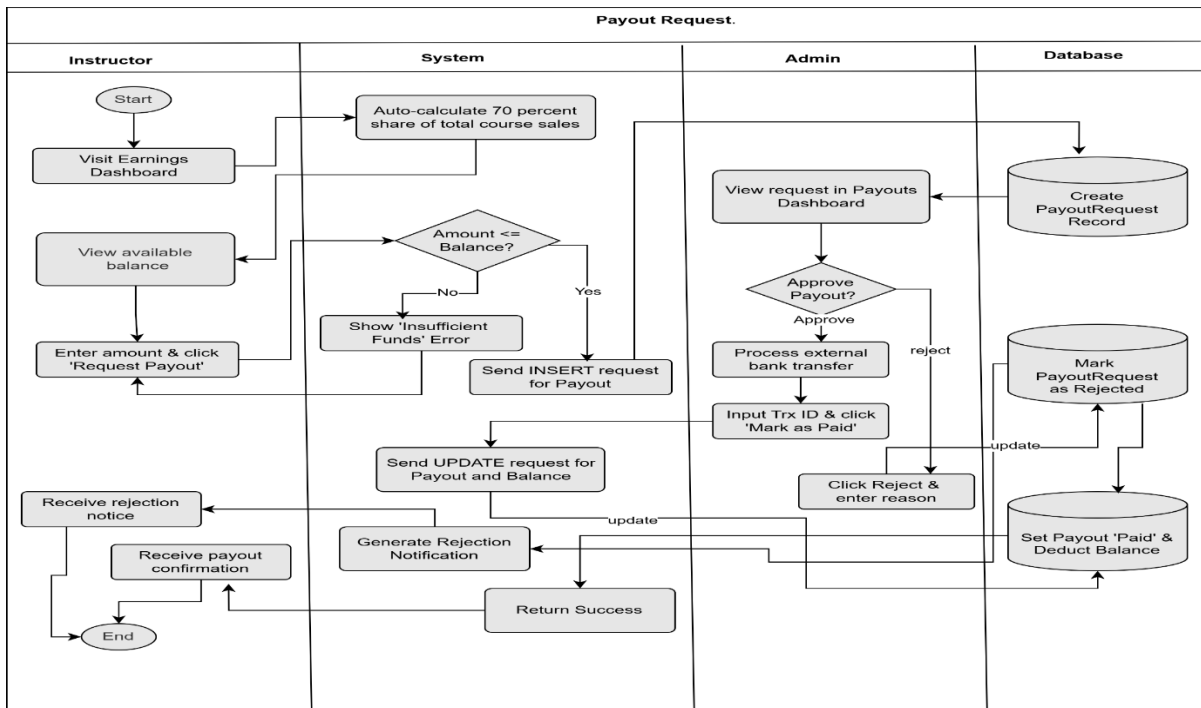


Figure 4.16: Swim-lane Diagram for Payout Request

4.4 Class Diagram

The Class Diagram is the basic structure of the database of the platform. It is a complete visual representation of the backend architecture which shows all Models in the database and the data type and constraints in them (Primary and Foreign Keys).

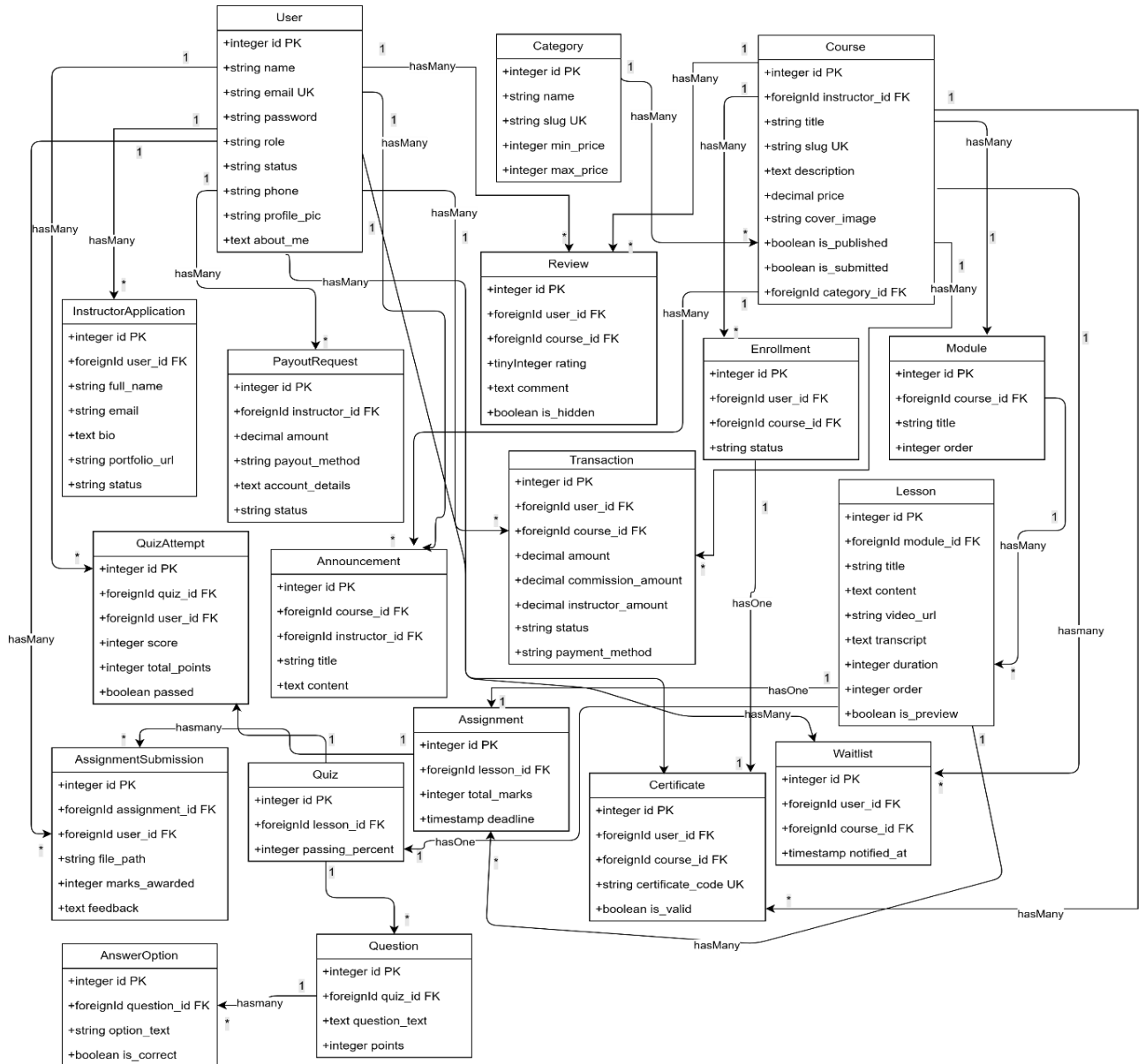


Figure 4.17: Class Diagram

Chapter 5. Project Management

5.1 Project Management

Project management is extremely essential in the successful completion of software development within the allocated time frame, budget, and quality level. In the development process of the EduBridge learning and teaching platform, an evaluation of risks has been carried out from the perspective of technology, security, operation, and financial areas. As EduBridge uses external APIs (Groq for the AI tutor) and third party financial gateways (Stripe/SSLCommerz), risk management has been used in its full capacity in order to avoid any adverse effects on the project. This chapter elaborates on risk identification, risk analysis, risk planning, and the overall risk mitigation monitoring, and management (RMMM) plan.

5.2 Risk Identification

Risk identification is the initial step of project management which includes identification of potential threats that may cause harm to the system functionality or security. For EduBridge, these have been identified as six core domains.

Table 5.1: Comprehensive Risk Identification

Risk Type	Possible Risk
Technical	• Third-party AI failure (Groq API rate limits or extended downtime). • Asynchronous payment webhook synchronization failure • System downtime or data loss during CI/CD production deployments.
Security	• Unauthorized access or piracy of premium video content. • Financial manipulation exploiting the 70/30 instructor payout logic. • Application vulnerabilities such as Cross-Site Scripting (XSS) or SQL Injection.

Operational	• AI Tutor delivering irrelevant answers. • Instructors adding illegal or copyrighted content in the course. • High dropout rates among learners because of difficult navigation.
Infrastructure	• Server storage exhausted because of high-resolution video upload. • Deadlocks in database during quiz submissions. • Memory exhaustion in Laravel queue workers.
Requirement	• Scope creep regarding the AI Retrieval-Augmented Generation (RAG) features. • Misalignment of features between Instructor expectations and Admin controls.
Legal/Financial	• Student payment disputes or credit card chargebacks. • Non-compliance with data privacy standards regarding student information.

5.3 Risk Analysis

Once identified, risks are analyzed based on the probability of their occurrence and their potential severity (effect) on the EduBridge platform.

Table 5.2: Risk Probability and Impact Analysis

Risk	Probability	Effect
Payment webhook synchronization failure	Low	Very High
Payout logic calculation errors / exploitation	Low	Very High
Application vulnerabilities (XSS / SQLi)	Low	Very High
Unauthorized access to premium videos	Medium	High

Third-party AI failure (Groq API downtime)	Medium	High
Database bottlenecks during concurrent quizzes	Low	High
Instructors uploading improper content	Medium	High
Student payment disputes or chargebacks	Medium	High
Server storage limits exceeded from uploads	High	Medium
AI Tutor generating inaccurate context	Medium	Medium
Scope creep regarding AI features	High	Medium
Memory exhaustion in queue workers	Medium	Medium

5.4 Risk Planning

Risk planning outlines the immediate architectural and strategic approaches taken during the development phase to prevent or minimize the impact of the analyzed risks.

Table 5.3: Strategic Risk Planning

Risk	Strategy
Webhook / Payment Failure	Implement cryptographic signature verification for all webhooks; utilize database transactions (DB::transaction) to ensure rollbacks if enrollment fails mid-execution.
Payout Exploitation	Hardcode the 70/30 split logic strictly on the backend controller; require manual Admin review and approval before any external bank transfers occur.
Security Vulnerabilities	Utilize Laravel's built-in Eloquent ORM to prevent SQL injection; apply CSRF tokens on all forms and strictly sanitize/escape user inputs to prevent XSS.

Groq API Failure	Implement robust try-catch error handling; display a graceful "AI currently unavailable" UI message to learners instead of crashing the lesson page.
AI Hallucinations	Strictly implement the Retrieval-Augmented Generation (RAG) architecture to force the AI to answer <i>only</i> based on the specific lesson transcript fetched from the database.
Storage Overflow	Compress video assets upon upload; utilize scalable cloud storage (e.g., Amazon S3) rather than local server storage.
Database Bottlenecks	Optimize database indexing on the quiz_attempts and notifications tables; utilize Laravel queues for heavy asynchronous processing tasks.
Improper Content	Implement a mandatory Admin moderation queue (Draft → Pending → Published) before any instructor course becomes visible to the public.
Disputes/Chargebacks	Maintain detailed, timestamped transaction logs; clearly state refund policies on the checkout portal before purchase.

5.5 Risk Monitoring

Continuous risk monitoring is needed to keep mitigation measures effective during the platform's entire lifecycle.

- **Code and System Review:** Perform regular code review and check the status of the upcoming features against the schedule.
- **Performance Monitoring:** Use application performance monitoring tools (e.g., Laravel Telescope) to check slow database queries and monitor the memory used in background queues.
- **External APIs Performance:** Monitor failure rates and latency of API requests made to the Groq engine.

- **Money Flows Monitoring:** Monitor external payment gateway dashboards (Stripe/SSLCommerz) for webhook delivery failure notifications and disputes.
- **Storage Capacities Alerting:** Monitor system storage capacities and set up alerts in case server storage exceeds 80% usage.
- **Moderation Queues:** Check “Pending Applications” and “Review Moderation” dashboards regularly to avoid administrative bottleneck risks.

5.6 RMMM Plan

Risk Mitigation, Monitoring, and Management (RMMM) plan covers a matrix of prevention, monitoring, and management steps needed to mitigate risks that are critical to the platform.

Table 5.4: The RMMM Chart

Risk	Mitigation Strategy	Monitoring Strategy	Management (Contingency)
Payment Webhook Failure	Verify signatures cryptographically. Ensure idempotent webhook processing to prevent duplicate charges.	Monitor server error logs and Stripe/SSLCommerz dashboard for HTTP 500 delivery failures.	Manually reconcile the database <code>transactions</code> table with gateway logs. Trigger manual enrollment overrides if the payment cleared.
GroqAPI Downtime	Build fault-tolerant API service classes with strict timeout	Track API response times and exception logs via Laravel's logging channels.	Temporarily disable the "Ask AI Tutor" button. Notify-users of maintenance and switch

	limits (e.g., 10 seconds).		to a fallback LLM model if available.
Payout Split Exploitation	Calculate the 70% share purely via backend logic. Never trust client-side data.	Admin manually reviews requested amounts against total course sales before processing.	Reject the payout request in the database. Suspend the offending instructor account and conduct a full financial audit.
Unauthorized Video Access	Protect video streaming routes using authentication middleware and signed, time-expiring URLs.	Monitor access logs for unusual IP activity or exceptionally high bandwidth consumption.	Revoke user session tokens immediately. Ban the offending IP addresses and dynamically cycle storage access keys.
Storage Overflow	Enforce maximum file size limits on frontend uploads.	Set automated system alerts when disk space reaches threshold capacity (80%).	Automatically trigger the deletion of temporary/orphaned files. Scale cloud storage tiers dynamically to prevent outages.
Inappropriate Course Content	Require Admin approval for all newly created courses before they achieve 'Published' status.	Admins routinely check the "Pending Courses" and "Review Moderation" queues.	Reject the course with mandatory feedback. Suspend-repeat offenders, permanently delete the flagged media.

Data Breach / Vulnerability	Utilize modern framework protections (CSRF, Eloquent PDO). Enforce strong password hashing (Bcrypt).	Run automated vulnerability scanners and monitor for abnormal login attempts.	Force a global password reset. Patch the identified vulnerability and notify users in compliance with data privacy regulations.
-----------------------------	--	---	---

Chapter 6. Project Planning

6.1 System Project Estimation

Prior to embarking upon the development process of EduBridge learning and teaching system, a proper project estimation was carried out. It is an important aspect of project management that helps allocate resources effectively and finish complex modules such as the Groq AI Tutor and Stripe/SSLCommerz payment gateways before the deadline set by the educational institution. It involves the four components of effort, cost, time, and team.

6.1.1 Effort Estimation (in Person-Hours)

The total effort was calculated based on the functional complexity of the platform's distinct modules. The breakdown reflects the heavy engineering required for the backend architecture and third-party API integrations.

Table 6.1: Effort Estimation by Module

Module / Phase	Estimated Hours
Requirements Gathering & Scope Definition	100 hours
UI/UX Wireframing & System Design	150 hours
Database Architecture & Schema Design	120 hours
Backend Development (Laravel Core)	350 hours
Frontend Development (Dashboard & Course Player)	300 hours
AI Integration (Groq API & RAG Architecture)	150 hours
Financial Integration (Stripe, SSLCommerz)	100 hours
Testing (Unit, Integration, & UAT)	200 hours
Deployment & Server Configuration	50 hours
Project Documentation & Thesis Formatting	100 hours
Buffer (Contingency ~3%)	52 hours
Total Estimated Effort	1,672 hours

6.1.2 Cost Estimation (BDT)

In order to make a comparison with commercial development of similar nature in terms of cost, the cost is estimated based on a standard entry level engineering rate for a team of developers in Bangladesh. Assuming a standard commercial rate of 400 BDT/hour:

Calculation: Total Estimated Hours $\times$ Hourly Rate

Total Cost: 1,672 hours $\times$ 400 BDT = 668,800 BDT

6.1.3 Time Estimation (Calendar Weeks)

The timeline for the project has been structured taking into consideration the traditional final year student's academic calendar and on the basis that the development takes place simultaneously.

Table 6.2: Time Estimation by Project Phase

Phase	Duration
Planning, Requirements & Scope	2 weeks
Design (UI, System Flow, Database Schema)	2 weeks
Core Development (Frontend, Backend, APIs)	6 weeks
Testing & Bug Fixing (QA & UAT)	2.5 weeks
Deployment, Documentation & Handover	1.5 weeks
Total Project Duration	~14 weeks

6.1.4 Team Estimation

To ensure that the deadline of 14 weeks and the man-hours of 1,672 hours are met, there should be an organization devoted to the development process. In this case, the core team of 3 people performs these duties well.

Table 6.3: Team Resource Allocation

Role	Number of Resources
Project Manager / Lead Architect	1
Backend & API Integration Developer	1
Frontend Developer & UI/UX Designer	1
Database Administrator	1 (Shared role)
QA / Test Engineer	1 (Shared role)

6.2 Function Matrix

Function Matrix groups all functions of the EduBridge system into different logical modules. Function Matrix identifies the particular functions that execute the system's business logic and relate directly to system controllers, user interfaces, and external API integration.

Table 6.4: Function Matrix describing all modules and functions of the EduBridge system.

Module Name	Function Name	Description
Authentication & Registration	registerUser()	Registers a new user with hashed credentials and assigns a default 'learner' role.
	loginUser()	Authenticates user credentials and provisions a secure session token.
	logoutUser()	Terminates the active session and clears authentication tokens.
	resetPassword()	Generates a secure recovery link to allow users to reset forgotten passwords.
User Profile Management	viewProfile()	Retrieves and displays a user's personal details, bio, and platform activity.
	updateProfile()	Allows users to edit their personal information and contact details.
	uploadProfilePicture()	Processes and securely stores user avatar images.
	submitInstructorApp()	Allows learners to submit an application and portfolio to request instructor privileges.

Course Catalog & Browsing	browseCourses()	Retrieves all published courses for display in the public catalog.
	filterCourses()	Filters the course catalog dynamically based on category and price range.
	viewCourseDetails()	Displays specific metadata, syllabus, and instructor information for a selected course.
	joinWaitlist()	Adds a learner to a notification queue if a specific course is currently unavailable.
Course Content Creation	createCourse()	Allows instructors to define course metadata, pricing, and upload cover images.
	updateCourseInfo()	Enables instructors to edit existing course descriptions and status (Draft/Pending).
	createModule()	Defines a structural chapter/section within a specific course.
	addLesson()	Allows instructors to upload video URLs, lesson text, and lesson transcripts.
Interactive Learning & AI	viewLesson()	Renders the video player and text content for an enrolled learner.
	askAITutor()	Transmits learner queries alongside lesson transcripts to the Groq API (RAG architecture) to generate real-time contextual answers.
Assessments (Quizzes)	createQuiz()	Allows instructors to define quiz parameters, including the passing percentage.
	addQuestion()	Adds multiple-choice questions and defines correct/incorrect options.
	attemptQuiz()	Loads interactive quiz questions and tracks the learner's answers in real-time.
	autoGradeQuiz()	Calculates the percentage score of a submitted quiz and updates the database with a pass/fail status.
	viewQuizResults()	Displays the final score and highlights correct/incorrect answers for the learner.
Assessments (Assignments)	createAssignment()	Allows instructors to set project requirements, maximum marks, and submission deadlines.
	submitAssignment()	Facilitates secure file uploads for learners submitting project files.

	gradeAssignment()	Allows instructors to download learner submissions, input numeric grades, and provide text feedback.
Financials & Enrollments	initiateCheckout()	Redirects the user to the Stripe or SSLCommerz payment portal with cart payload.
	processPaymentWebhook()	Securely catches asynchronous success payloads from gateways to finalize transactions.
	enrollUser()	Grants database access rights linking a learner to a specific course upon payment.
	viewTransactionHistory()	Displays a ledger of past purchases and payment statuses for the user.
Instructor Payouts	calculateRevenueSplit()	Automatically calculates the 30% platform commission and 70% instructor share.
	viewEarningsDashboard()	Displays accumulated instructor revenues and current available balances.
	requestPayout()	Enables instructors to request a bank withdrawal of their calculated earnings balance.
Social & Communication	submitReview()	Allows enrolled learners to leave a 1-5 star rating and text feedback on a course.
	postAnnouncement()	Broadcasts targeted messages from instructors to enrolled learners.
	viewAnnouncements()	Displays a feed of chronological notifications on the learner's dashboard.
Certification System	generateCertificate()	Dynamically generates a PDF certificate with a cryptographic code upon 100% completion.
	downloadCertificate()	Allows learners to retrieve and save their generated PDF certificates.
	verifyCertificate()	Allows external users (e.g., employers) to publicly verify the authenticity of a certificate code.
System Administration	viewAdminDashboard()	Displays global system statistics, revenue totals, and pending user actions.
	moderateCourse()	Allows administrators to review, approve, or reject instructor-created courses.

	approveInstructorApp()	Allows administrators to grant 'Instructor' role privileges to pending applicants.
	auditPayouts()	Allows administrators to review, approve, and mark instructor payout requests as 'Paid'.
	suspendUser()	Allows administrators to revoke platform access for users violating terms of service.

6.3 Identifying Complexity

6.3.1 Identifying Complexity of Transition Function

Transition functions denote the data that flows through the system boundary. External Input (EI) deals with input data, External Output (EO) deals with output data, and External Inquiries (EQ) deal with data retrieval.

Table 6.5: Identifying complexity (Transition Function)

Transition Function	Fields / File Involved	FTRs	DETs
User Registration (EI)	Fields – Name, Email, Password, Role File Name – users	1	4
Submit Instructor App (EI)	Fields – Name, Email, Bio, Portfolio File Name – instructor_applications	1	4
Create Course (EI)	Fields – Title, Slug, Desc, Price, Image File Name – courses, categories	2	5

Create Module & Lesson (EI)	Fields – Title, Order, Content, Video, Transcript File Name – modules, lessons	2	5
Create Quiz (EI)	Fields – Passing %, Q_Text, Points, Options, Is_Correct File Name – quizzes, questions, answer_options	3	5
Process Payment (EI)	Fields – Amount, Status, Method, Signature File Name – transactions, enrollments	2	4
Submit Assignment (EI)	Fields – File_Path File Name – assignment_submissions	1	1
Grade Assignment (EI)	Fields – Marks_Awarded, Feedback File Name – assignment_submissions	1	2
Request Payout (EI)	Fields – Amount, Method, Account_Details File Name – payout_requests	1	3
Submit Review (EI)	Fields – Rating, Comment File Name – reviews	1	2

Post Announcement (EI)	Fields – Title, Content File Name – announcements	1	2
Admin Revenue Dash (EO)	Fields – Calc 30% Platform Fee, Total Orders File Name – transactions, courses	2	4
Auto-Grade Quiz (EO)	Fields – Calc Total Score, Pass/Fail File Name – quiz_attempts, quizzes	2	4
Generate Certificate (EO)	Fields – Code, Valid_Flag, Course, User, Date File Name – certificates, courses, users	3	5
AI Tutor Response (EO)	Fields – Context_Prompt, Generated_Answer File Name – lessons (Groq API)	1	1
Instructor Earnings (EO)	Fields – Calc 70% Share, Balance File Name – transactions, payout_requests	2	3
User Login (EQ)	Fields – Email, Password File Name – users	1	2
Browse Catalog (EQ)	Fields – Keywords, Category, Price File Name – courses, categories	2	3

View Profile (EQ)	Fields – Name, Bio, Phone, Pic, Role File Name – users	1	5
Access Lesson (EQ)	Fields – Course_ID, Lesson_ID File Name – courses, lessons	2	2
Verify Certificate (EQ)	Fields – Certificate_Code File Name – certificates	1	1

6.3.2 Identifying Complexity of Data Function

The data functions stand for the logical data that is stored internally (ILF), and also external interfaces that the system interacts with (EIF).

Table 6.6: Identifying complexity (Data Function)

Data Function	Fields / File Involved	RETs	DETs
Users (ILF)	Fields – id, name, email, pass, role, status, phone, pic, bio, created_at	6	10
Courses (ILF)	Fields – id, inst_id, cat_id, title, slug, desc, price, img, pub, sub	6	11
Categories (ILF)	Fields – id, name, slug, min_price, max_price	1	5
Modules & Lessons (ILF)	Fields – id, course_id, title, order, content, video, transcript, duration	3	13

Quizzes & Questions (ILF)	Fields – id, lesson_id, passing_pct, text, points, option_text, is_correct	3	11
Assignments (ILF)	Fields – id, lesson_id, marks, deadline, file, awarded, feedback	2	10
Transactions (ILF)	Fields – id, user_id, course_id, amt, com_amt, inst_amt, status, method	2	8
Enrollments (ILF)	Fields – id, user_id, course_id, status	3	4
Certificates (ILF)	Fields – id, user_id, course_id, code, is_valid	2	5
Instructor Apps (ILF)	Fields – id, user_id, name, email, bio, portfolio, status	1	7
Payout Requests (ILF)	Fields – id, inst_id, amount, method, details, status	1	6
Payment Gateways (EIF)	Fields – Stripe/SSLCommerz Auth Keys, Payload Data	1	5
Groq AI Engine (EIF)	Fields – Inference Context, API Token, Model Params	1	4

6.3.3 Unadjusted Function Point Contribution (Transition Function)

Table 6.7: Unadjusted function point contribution (Transition Function)

Transition Functions	FTRs	DETs	Complexity	UFP
User Registration	1	4	Low	3

Submit Instructor App	1	4	Low	3
Create Course	2	5	Low	3
Create Module & Lesson	2	5	Low	3
Create Quiz	3	5	Average	4
Process Payment	2	4	Low	3
Submit Assignment	1	1	Low	3
Grade Assignment	1	2	Low	3
Request Payout	1	3	Low	3
Submit Review	1	2	Low	3
Post Announcement	1	2	Low	3
Admin Revenue Dash	2	4	Low	4
Auto-Grade Quiz	2	4	Low	4
Generate Certificate	3	5	Average	5
AI Tutor Response	1	1	Low	4
Instructor Earnings	2	3	Low	4
User Login	1	2	Low	3
Browse Catalog	2	3	Low	3
View Profile	1	5	Low	3
Access Lesson	2	2	Low	3
Verify Certificate	1	1	Low	3
Total				70

6.3.4 Unadjusted Function Point Contribution (Data Function)

Table 6.8: Unadjusted function point contribution (Data Function)

Data Function	RETs	DETs	Complexity	UFP
Users (ILF)	6	10	High	15
Courses (ILF)	6	11	High	15
Categories (ILF)	1	5	Low	7
Modules & Lessons (ILF)	3	13	Average	10
Quizzes & Questions (ILF)	3	11	Average	10
Assignments (ILF)	2	10	Low	7
Transactions (ILF)	2	8	Low	7
Enrollments (ILF)	3	4	Low	7
Certificates (ILF)	2	5	Low	7
Instructor Apps (ILF)	1	7	Low	7

Payout Requests (ILF)	1	6	Low	7
Payment Gateways (EIF)	1	5	Low	5
Groq AI Engine (EIF)	1	4	Low	5
Total				109

6.3.5 Performance and Environmental Impact

Table 6.9: Performance and Environmental Impact

GSC	TDI
Data Communication (Webhooks/APIs)	4
Distributed Data Processing	3
Performance Objectives	4
Heavily Used Configuration	3
Transaction Rate	4
Online Data Entry	5
End User Efficiency	5
Online Update	4
Complex Processing (AI / 70-30 Split)	5
Reusability	3
Installation Ease	3
Operational Ease	4
Multiple Sites	2
Facilitate Change	3
Total Degree of Influence (TDI) (Range 0-70)	52

Calculations:

Value Adjustment Factor (VAF) = $0.65 + (0.01 \times \text{TDI})$

= $0.65 + (0.01 \times 52) = 0.65 + 0.52 = 1.17$

Unadjusted Function Point (UFP) = Data Functions + Transaction Functions

$$= 109 + 70 = 179$$

Adjusted Function Point Count (AFP) = UFP $\times$ VAF

$$= 179 \times 1.17 = 209.43 \approx 209$$

Effort Estimation:

Table 6.10: Estimated Development Effort Rates by Technology Stack

Language / Framework	Hours per function point
HTML / Tailwind CSS	4.0
JavaScript	6.0
PHP (Laravel) / MySQL	8.0

Using the blended backend/database rate of 8.0 hours per function point:

Effort for EduBridge System:

$$= 209 \text{ AFP} \times 8.0 \text{ hours}$$

$$= 1,672 \text{ person hours}$$

$$= 1,672 \text{ person hours} / 8 \text{ hours per day}$$

$$= 209 \text{ person days}$$

$$= 209 \text{ person days} / 3 \text{ [persons in the core team]}$$

$$= 69.6 \text{ days per person}$$

$$= 69.6 \text{ days} / 22 \text{ working days per month}$$

$$= \sim 3.16 \text{ months per person (Aligns exactly with the } \sim 14\text{-week project duration schedule).}$$

6.4 Project Scheduling Chart

Development is scheduled to take place throughout a 4 month period. This is made possible by breaking down the development process into different milestones, making use of parallel development to be as efficient as possible. For instance, the frontend wireframe is developed

at the same time as the backend of the app is developed, and the integration of Stripe and Groq APIs happens together with writing the assessment.

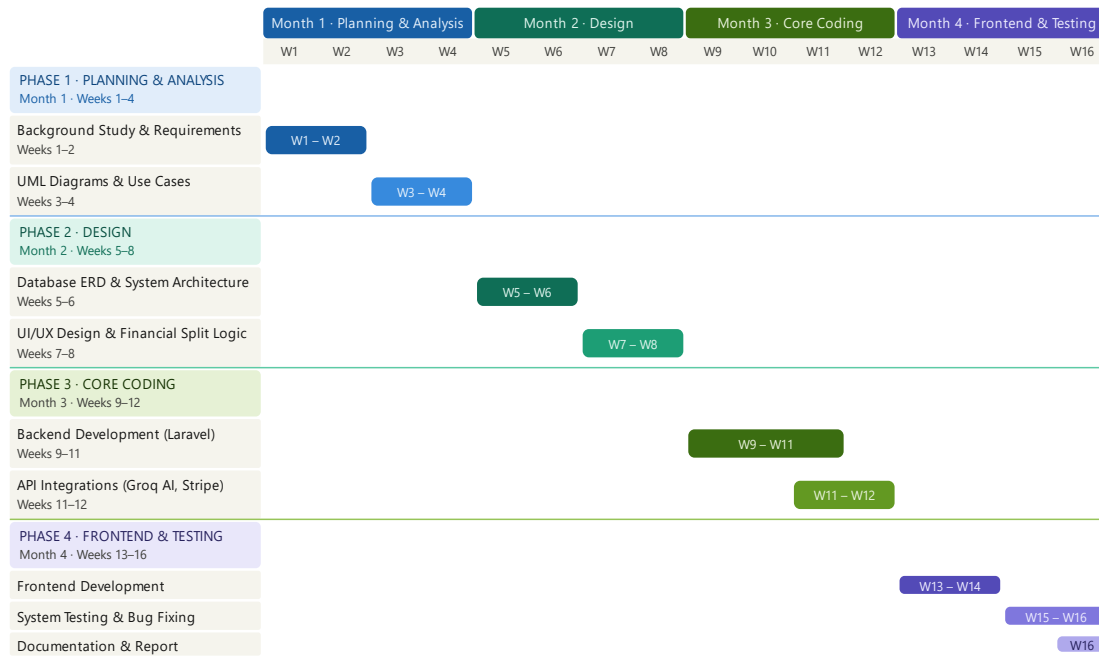


Figure 6.1: Project scheduling chart

6.5 Accounts Table

The accounts table presents an exhaustive business budget estimate for developing the EduBridge platform. This estimate includes the salaries of personnel based on organizational working hours, depreciation of hardware, software licensing fees, and workspace costs.

6.5.1 Personnel Cost

To estimate the personnel cost, the standard organizational working hours in a month should be estimated.

- Number of days in a year = 365
- Number of government holidays in a year = 24
- Number of weekly holidays in a year = 52
- Total number of working days in a year = $365 - (52 + 24) = 289$ days.

- Total number of working days per month = $289 / 12 = 24.08$ days.
- Organizational working hours per day = 8 hours.
- **Organization working hours per month** = $24.08 \times 8 = 192.64$ hours.

Based on a 4-month (16-week) development schedule for the 3-person core team, the personnel cost is estimated as follows:

Table 6.11: Personnel Cost Estimation

Particulars	Monthly Salary (BDT)	Total Cost for 4 Months (BDT)
Project Manager & Lead Architect	25,000.00	100,000.00
Backend & API Developer	25,000.00	100,000.00
Frontend & UI/UX Designer	20,000.00	80,000.00
Total Personnel Cost	70,000.00 /=	280,000.00 /=

6.5.2 Expected Hardware Cost

Hardware costs do not involve the total price of the hardware but only the depreciation cost used during the development process of the project.

Assuming a standard hardware lifespan of 15 years:

- The depreciation percentage per year is: $1 / 15 \Rightarrow 6.67\% = 0.0667$

Table 6.12: Expected Hardware Cost Estimation

Hardware Item	Original Value (BDT)	Depreciation Calculation	Cost (BDT)
Development Computers (3 Units)	150,000.00	$(150,000 \times 0.0667)$	10,005.00
Testing Devices (Smartphones)	40,000.00	$(40,000 \times 0.0667)$	2,668.00
Network Router & Printer	8,000.00	$(8,000 \times 0.0667)$	533.60

Total Hardware Cost			13,206.60/=
----------------------------	--	--	--------------------

6.5.3 Expected Software Cost

Unlike regular software lines, the EduBridge platform needs certain cloud infrastructures, domain settings, and API permissions to operate properly during the 4 months of development and testing.

Table 6.13: Expected Software Cost Estimation

Component Service	Description	Estimated Cost (BDT)
Application Hosting	DigitalOcean / AWS VPS for backend (4 months)	8,000.00
External APIs & AI	Groq API token usage & API testing limits	4,500.00
Domain & Security	.com Domain registration & SSL Certificate	2,500.00
Total Software Cost		15,000.00 /=

6.5.4 Expected Other Cost

Costs related to logistics and operations are incurred to keep the development environment alive.

Table 6.14: Expected Other Cost Estimation

Expense Type	Cost (BDT)
Workspace Rent	4,000.00
Furniture	6,000.00
Electricity Bill	1,500.00
Internet & Communication	1,000.00

Miscellaneous / Extra	1,000.00
Total Other Cost	13,500.00 /=

6.5.5 Total Accounts Summary

From the above financial breakdowns, it is clear that the following table below presents the total financial estimates needed to implement the EduBridge project.

Table 6.15: Total Accounts Summary

Particulars	Total Cost (TK)
Salary (4 Months)	
• Project Manager	100,000.00
• Backend Developer	100,000.00
• Frontend Designer	80,000.00
Subtotal Salary	280,000.00 /=
Hardware Cost (Depreciation)	
• Computers	10,005.00
• Testing Devices	2,668.00
• Router & Printer	533.60
Subtotal Hardware	13,206.60 /=
Software Cost	
• Hosting, APIs, and Domains	15,000.00
Subtotal Software	15,000.00 /=
Other Costs	
• Rent, Electricity, Internet, Furniture	13,500.00
Subtotal Other	13,500.00 /=
Grand Total Cost	321,706.60 /=

Chapter 7. Designing

7.1 Interface Design

The interface design of the EduBridge platform has been developed in such a way that it can cater to three types of users: Learner, Instructor, and Administrator. The design of each interface is such that it is extremely intuitive, free from distractions, and very responsive.

7.1.1 Learner (Student) Interfaces

The interface for the Learner is totally centered on learning courses.

A. User Registration and Login Page

- Purpose: Authenticate learners to access educational services.
- Features: Input fields for Name, Email, Password, and Role selection. Strict validation input data validation and secure session token creation.

B. Public Course Catalog & Discovery

- Purpose: Allow learners to browse and filter available educational content.
- Features: Dynamic grid displaying approved and published courses. Search and filter functionalities based on category and price. Detailed course landing pages featuring descriptions, instructor bios, and public reviews.

C. Checkout & Waitlist Interface

- Purpose: Facilitate secure enrollment and payment processing.
- Features: Seamless integration with Stripe and SSLCommerz payment portals. A dynamic “Join Waitlist” button when courses are not available.

D. Dedicated Course Viewer & AI Integration

- Purpose: Provide a focused, distraction-free learning environment.
- Features: Video player integration alongside text content. Real-time progress tracking bars. An embedded chat interface for communication with the RAG AI tutor for contextual learning support.

E. Assessments & Certification Portal

- Purpose: Allow learners to test their knowledge and claim rewards.

- Features: Interactive multiple-choice quizzes with instant auto-grading feedback. Practical assignment file upload websites. Screen for viewing, verifying, and downloading the PDF certificate on completion of 100% of the course.

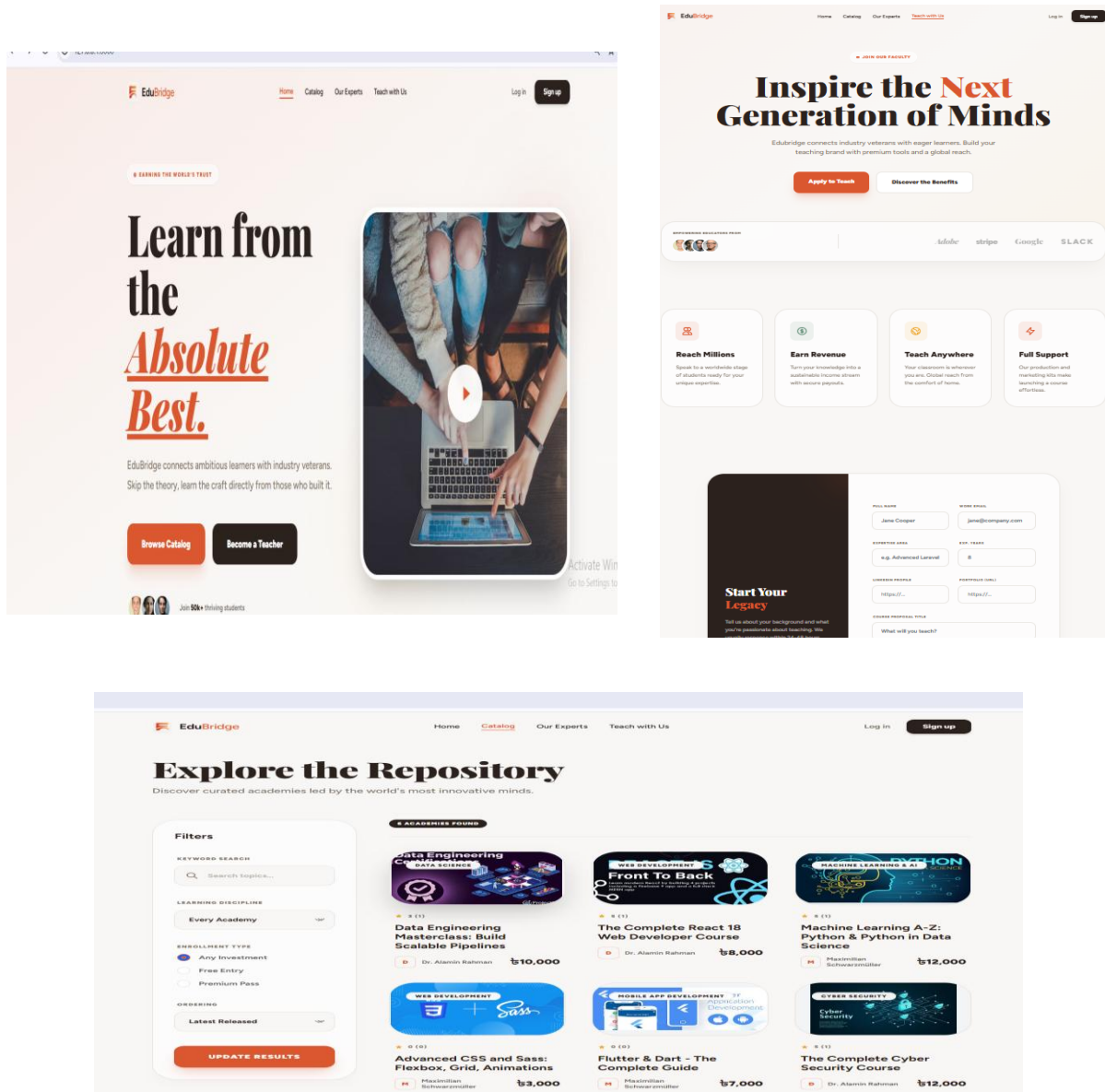


Figure 7.1: EduBridge public homepage and course catalog and Teacher application

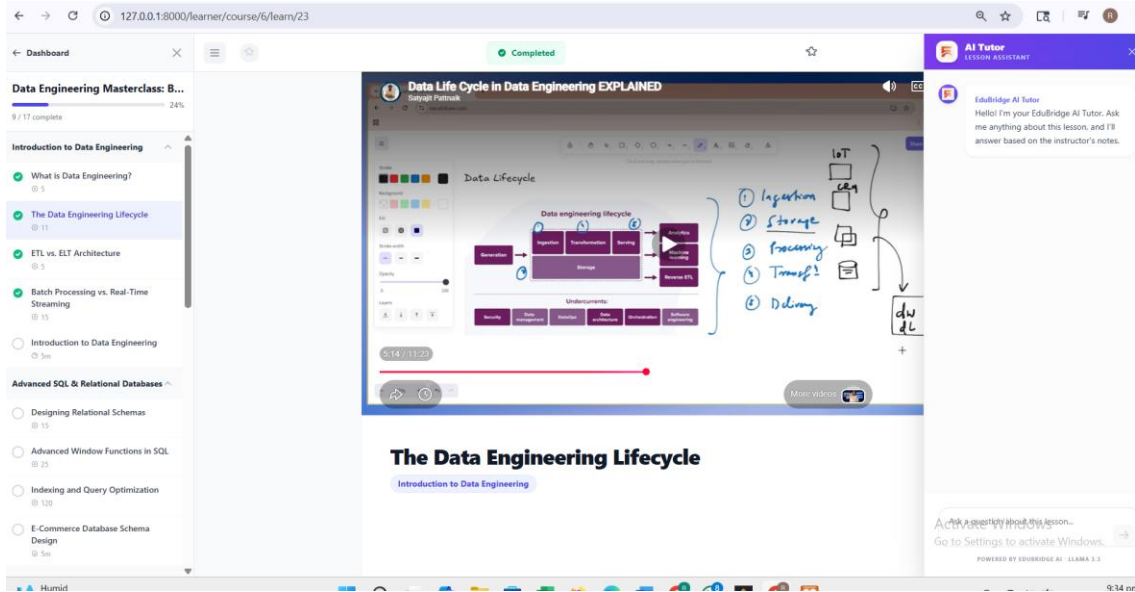
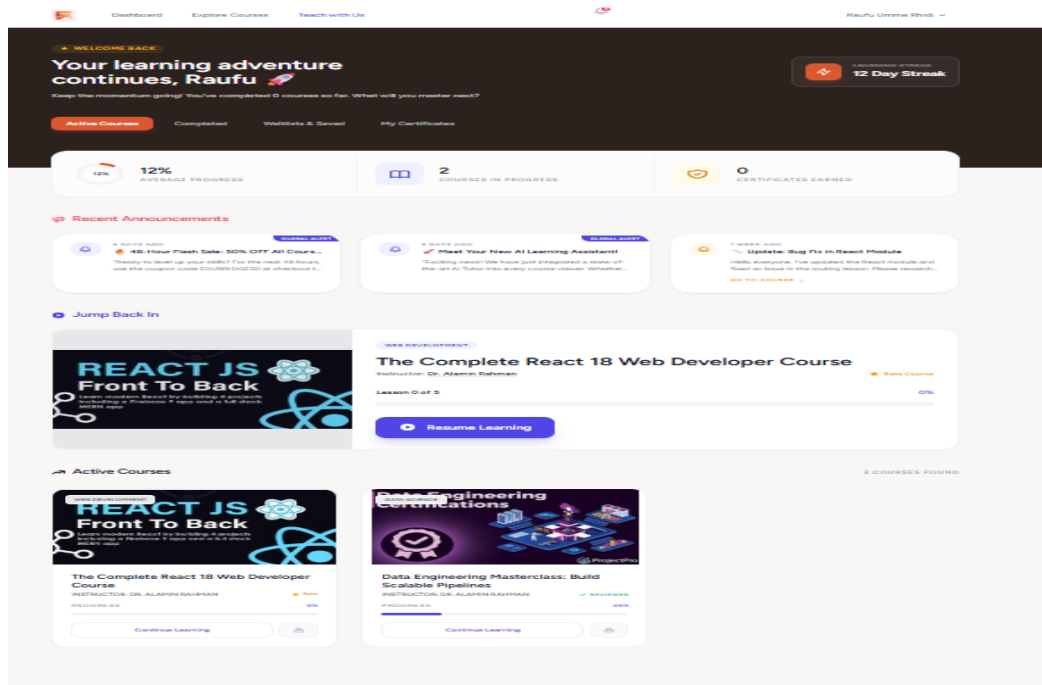


Figure 7.2: Dedicated Course Viewer interface integrating video lessons, syllabus tracking, and the real-time AI Tutor assistant.

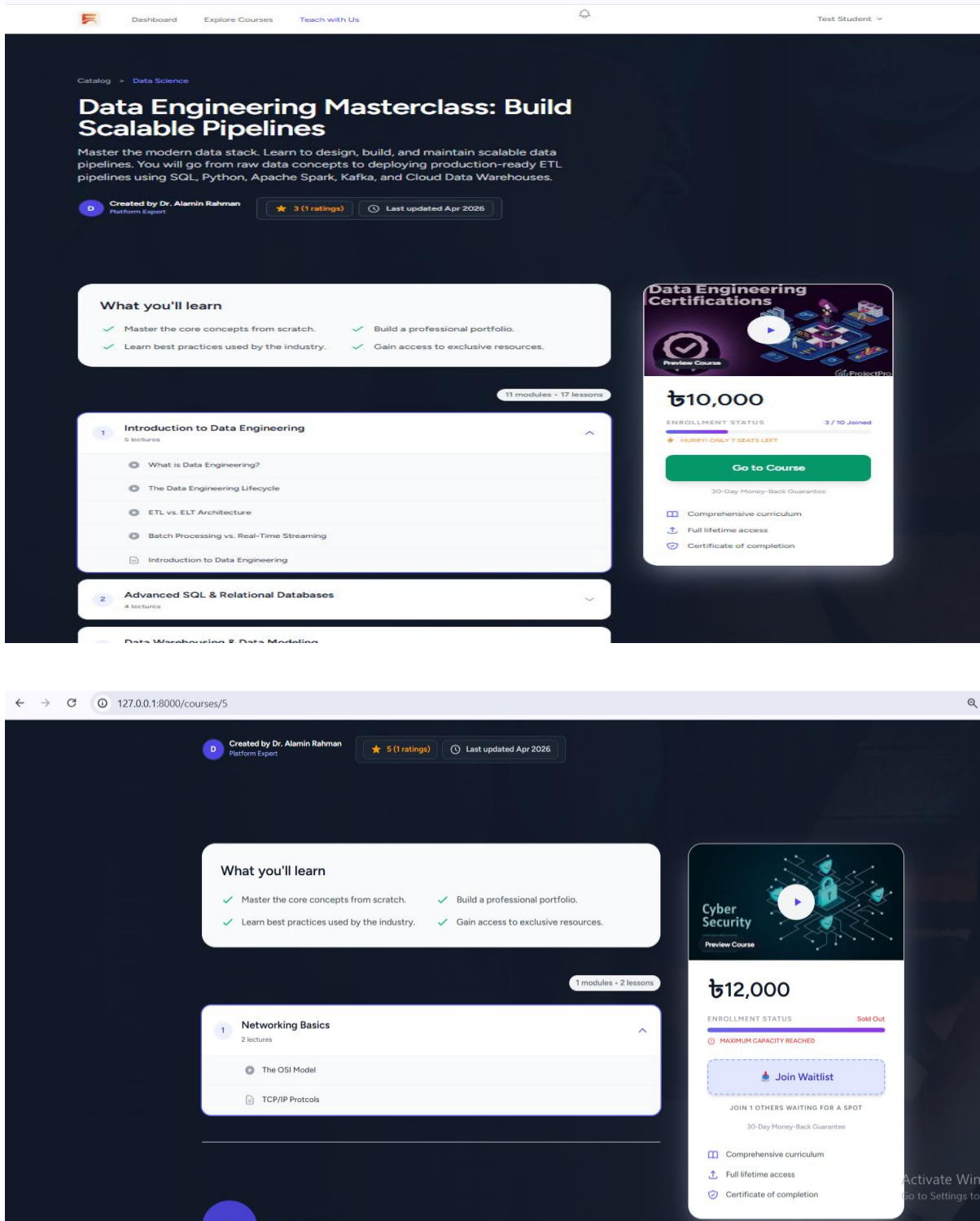


Figure 7.3: Course Catalog and Enrollment interface featuring dynamic pricing and the 'Join Waitlist' functionality.

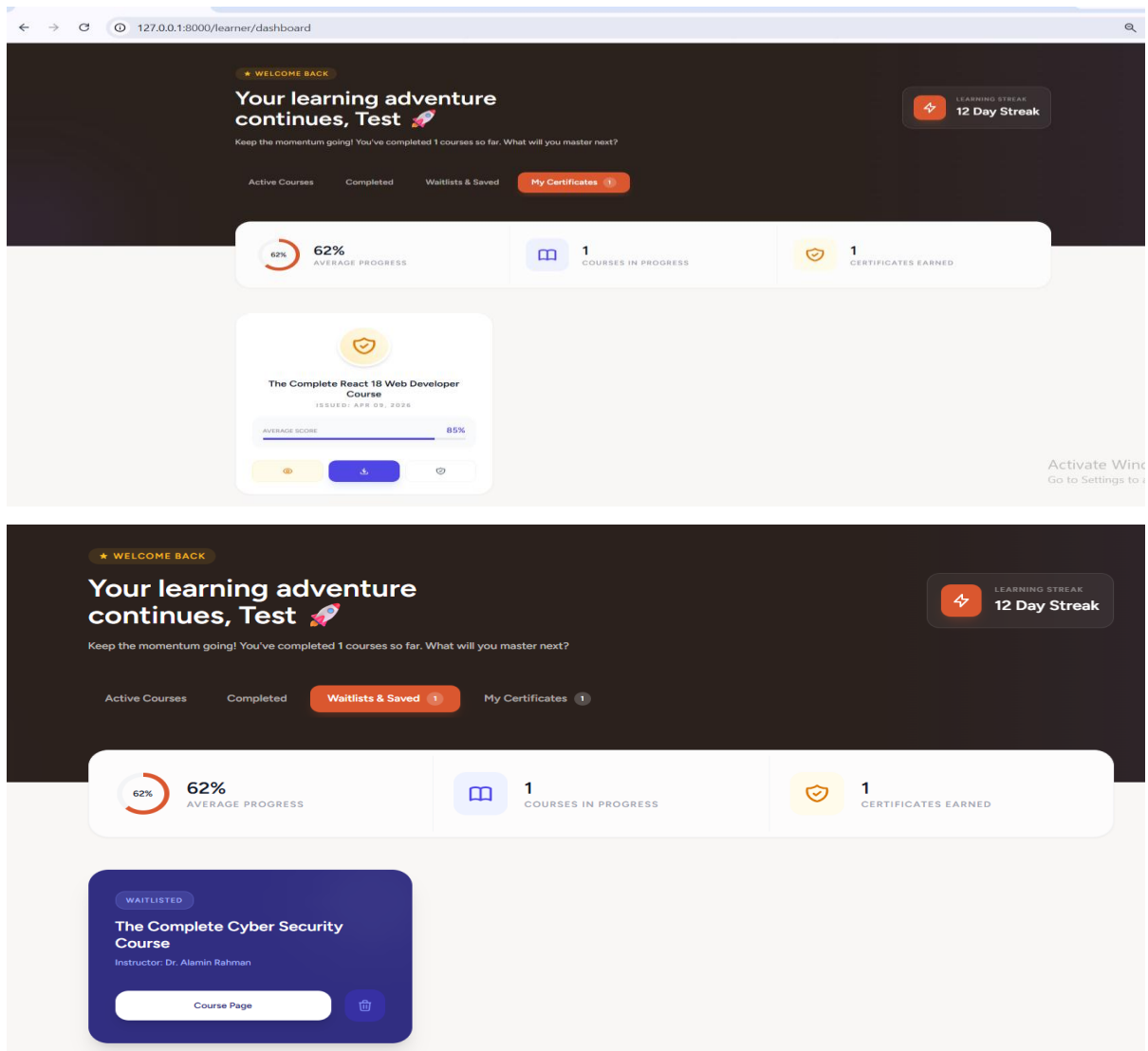


Figure 7.4: The Learner Dashboard displaying active course progress, learning streaks, and earned certificates.

7.1.2 Instructor (Teacher) Interfaces

Instructor dashboard serves as a “creator studio” that equips instructors with all necessary instruments for curriculum development and income management.

A. Creator Dashboard & Analytics

- **Purpose:** To give a high-level view of teaching performance and income.

- **Features:** Summary cards with the number of active courses, enrolled students, and earnings (calculated automatically at 70% of total sales).

B. Dynamic Course & Assessment Builder

- Purpose: Allow instructors to draft and structure educational content.
- Features: Multi-step form tools to create course metadata and upload cover images. Building Modules structure interfaces; interfaces to upload video/transcript Lessons. Tools to develop quiz questions, answers, and pass percentages.

C. Grading & Submissions Hub

- Purpose: Manage and evaluate student assignment submissions.
- Features: Dedicated notification dashboard for newly uploaded student files. Fields for entering grades and giving feedback in text form.

D. Payout Management Interface

- Purpose: Allow instructors to withdraw their accumulated earnings.
- Features: Display of current available balance. Forms to securely input bank/account Provide details and make "Payout Requests" to platform administrators.

E. Course Announcements Interface

- Purpose: Facilitate direct communication with enrolled students.
- Features: Text editor to draft and broadcast targeted announcements strictly to learners enrolled in specific courses.

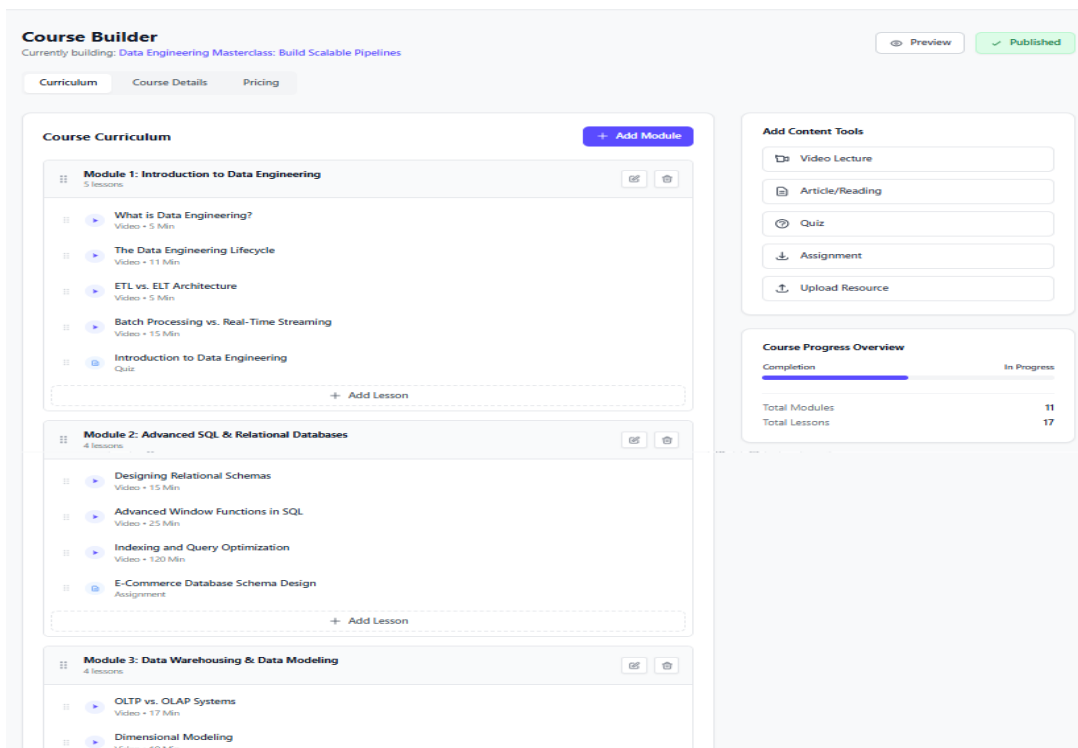
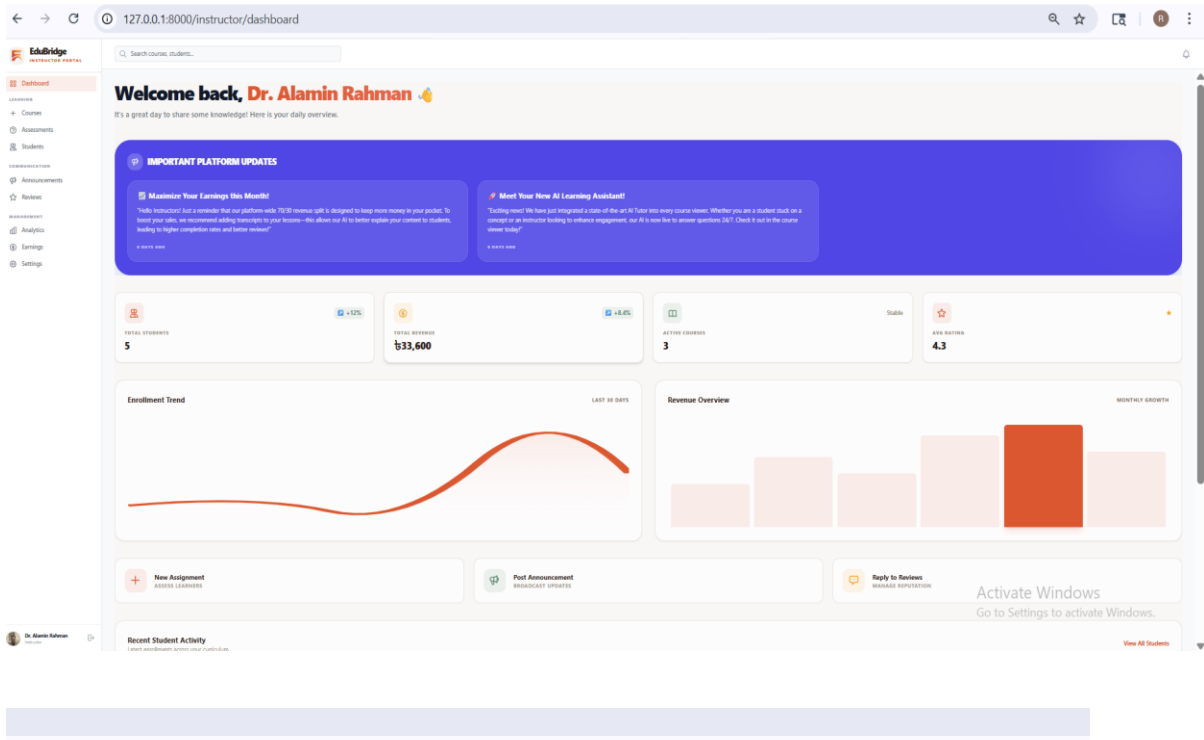


Figure 7.5: Instructor Creator Dashboard featuring high-level analytics, including enrollment trends and real-time revenue tracking.

Search courses, students...

Grading

Review and grade student submissions

PENDING REVIEW

0

GRADED

1

TOTAL SUBMISSIONS

1

AVG SCORE

0%

Pending (0)

Graded (1)

STUDENT	ASSIGNMENT	COURSE	SUBMITTED	FILE	ACTION
15 Test Student	Build a weather dashboard assignment	The Complete React 18 Web Developer Course	1 month ago	pGHyZ2UaKOYbMsOgq3PHVxYty6RHwIsq4IEIB4uM.pdf	Graded

Search courses, students...

Course Announcements

Broadcast important updates to your students

Post New Announcement

Target Course

-- Select Course --

Announcement Title

e.g., Assignment #2 Deadline Extended

Content Message

Write your message here...

Notification Notice:

This will send an email and a dashboard notification to all enrolled students immediately.

Broadcast Announcement

Announcement History

Maximize Your Earnings this Month!

PLATFORM WIDE

"Hello Instructors! Just a reminder that our platform-wide 70/30 revenue split is designed to keep more money in your pocket. To boost your sales, we recommend adding transcripts to your lessons—this allows our AI to better explain your content to students, leading to higher completion rates and better reviews!"

May 04, 2026

Meet Your New AI Learning Assistant!

PLATFORM WIDE

"Exciting news! We have just integrated a state-of-the-art AI Tutor into every course viewer. Whether you are a student stuck on a concept or an instructor looking to enhance engagement, our AI is now live to answer questions 24/7. Check it out in the course viewer today!"

May 04, 2026

Update: Bug Fix in React Module

THE COMPLETE REACT 18 WEB DEVELOPER COURSE

Hello everyone, I've updated the React module and fixed an issue in the routing lesson. Please rewatch the updated video if you faced any confusion earlier. Thanks for your patience!

Apr 26, 2026

Earnings Hub

MANAGING YOUR REVENUE INTELLIGENCE AND SPLITS.

FINANCIAL MASTERY

LIFETIME EARNINGS

₹33,600

AVAILABLE FOR PAYOUT

₹32,100

WITHDRAW

EARNINGS (MAY)

₹8,400

Revenue Trend

HISTORICAL FINANCIAL GROWTH (LAST 6 MONTHS)

Month	Revenue (₹)
Dec 2025	0
Jan 2026	0
Feb 2026	0
Mar 2026	0
Apr 2026	25000
May 2026	8400

Payout History

STATUS OF YOUR WITHDRAWALS

DATE	METHOD	AMOUNT	RECEIPT / TRXID	STATUS
Apr 11, 2026	BRASH	₹1,000	T2000800223	PAID
Apr 11, 2026	BRASH	₹500	WAIT...	PAID

Financial Statistics

TRANSPARENCY ON PLATFORM SPLITS (30/70)

COURSE NAME	SALES	GROSS (₹)	FEE (30%)	EARNING (70%)
The Complete React 18 Web Developer Course	2	₹16,000	-₹4,800	₹11,200
The Complete Cyber Security Course	1	₹12,000	-₹3,600	₹8,400

Activate Go to Sett

Figure 7.6: Instructor interfaces for Grading, Course Announcements, and Earnings Management.

7.1.3 Administrator Interfaces

The Admin Panel is a very safe and high-quality control panel meant for strict moderation of the platform, users, and finances.

A. Super Admin Dashboard

- Purpose: Provide a global overview of platform health and revenue.
- Features: Graphical representations of total registered users (split by role) and course statuses. Presentation of commission revenue earned by the platform amounting to 30%.

B. Course & Application Moderation Interface

- Purpose: Ensure platform quality and academic integrity.
- Features: A secure sandbox to preview "Pending" courses created by instructors, with publish or reject back to draft stage. A queue for reviewing and approving or rejecting applications for "Teach with Us" on the basis of user profiles and portfolios.

C. Financial & Payout Processing Interface

- Purpose: Monitor global transactions and process instructor withdrawals.
- Features: Ledger of all global transactions (Stripe/SSLCommerz). Tools to review Verify the instructor Payout Request funds, and manually confirm "Paid" with an external transaction ID.

D. User & Category Management

- Purpose: Maintain the organizational structure of the platform.
- Features: Full CRUD (Create, Read, Update, Delete) control over global course categories. To have the capability of seeing the user database, manual account creation and suspension/deletion of malicious users.

E. Review & Certificate Moderation

- Purpose: Monitor content available publicly and credentialing processes.

- Features: Options that can turn off spam or abusive student reviews on the public catalogue. Administrative tools for rendering certificates invalid in case of cheating.

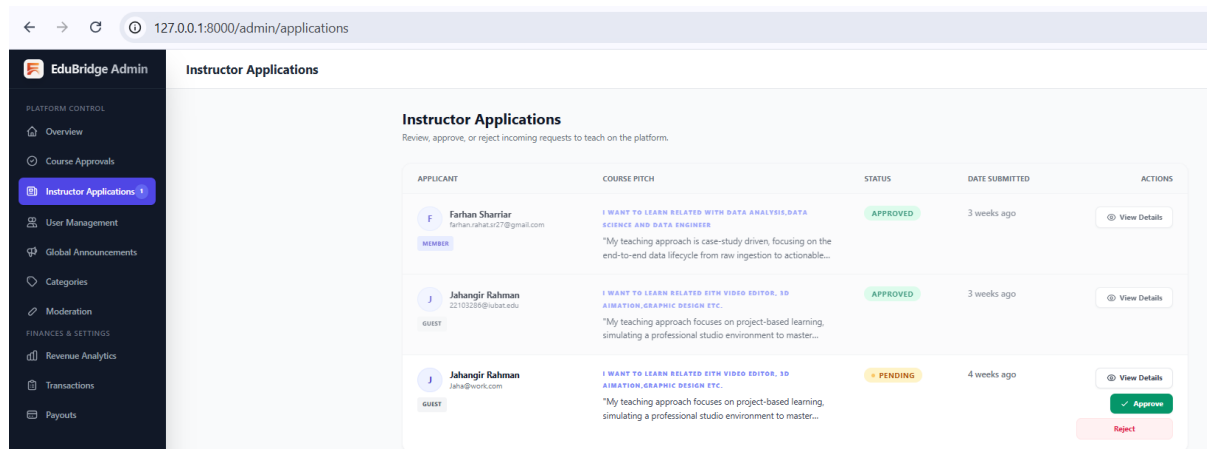
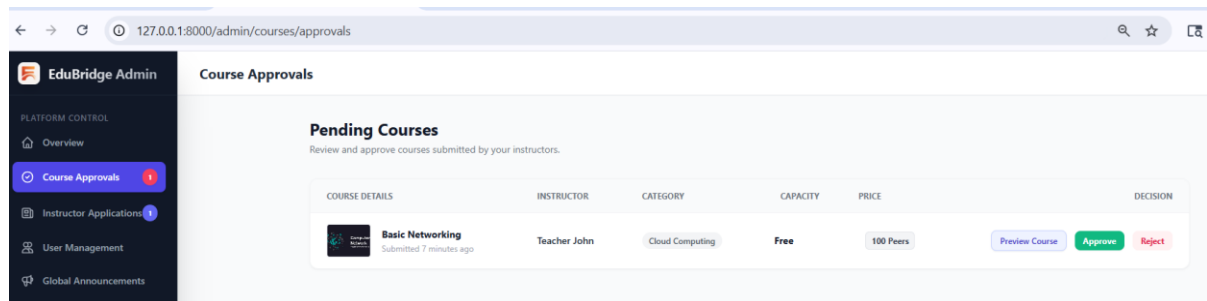
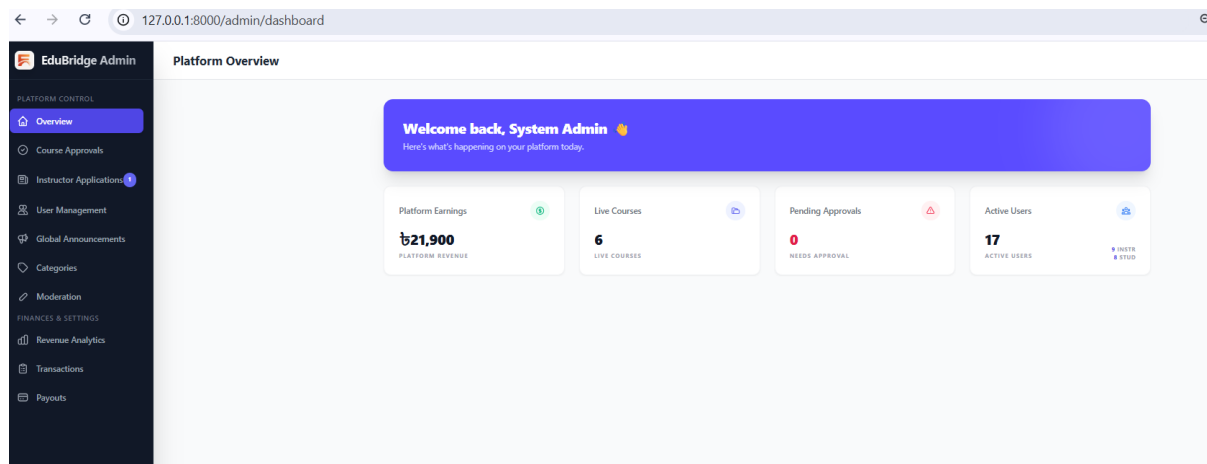


Figure 7.7: Admin interfaces for Platform Overview, Course Approvals, and Instructor Applications.

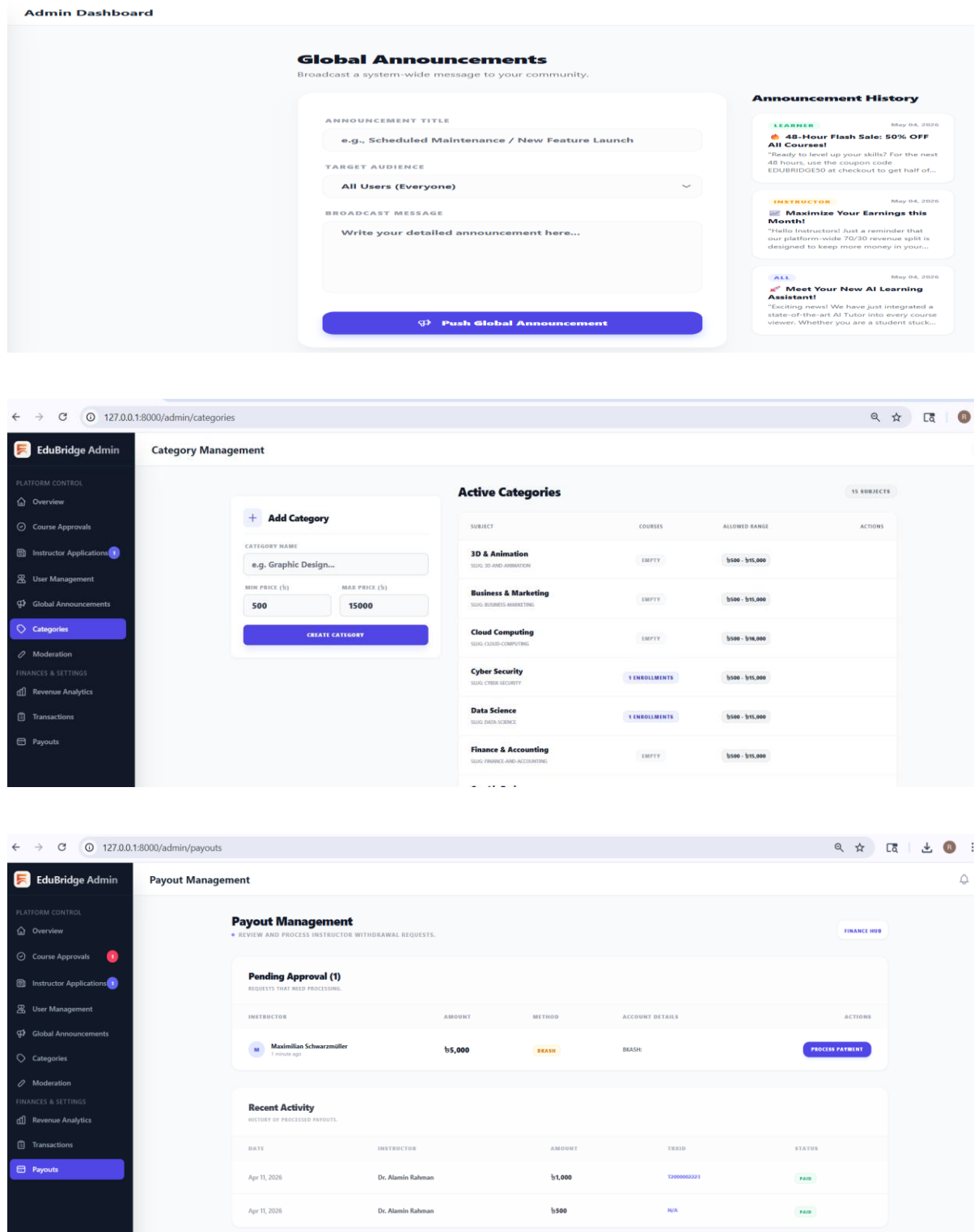


Figure 7.8: Admin interfaces for Global Announcements, Category Management, and Financial Payouts.

7.2 Data Flow Diagram (DFD)

Data Flow Diagram (DFD) represents the flow of information for the particular process/system. It uses certain standardized symbols like rectangle for external entity, split-box for process and open rectangle for data store, and depicts data inputs, outputs, data storage points and their logical paths to every destination point. In case of EduBridge online learning and teaching platform the data flow diagram is arranged in three hierarchical layers (Level 0, Level 1, and Level 2). Such an approach demonstrates how users' inputs, internal logic of the system and API services (Stripe and Groq AI) work together without being aware about the source code of the platform.

7.2.1 Context Diagram (Level 0 DFD)

The Level 0 Data Flow Diagram, also called as Context Diagram, shows all the system as one high-level process in general. It specifies primary external entities interacting with the system such as Learner, Instructor and System Admin from the user's side and Payment Gateways and Groq AI Engine from the service's side. Such DFD describes the absolute boundaries of the platform and the overall data payload going into and out of the system.

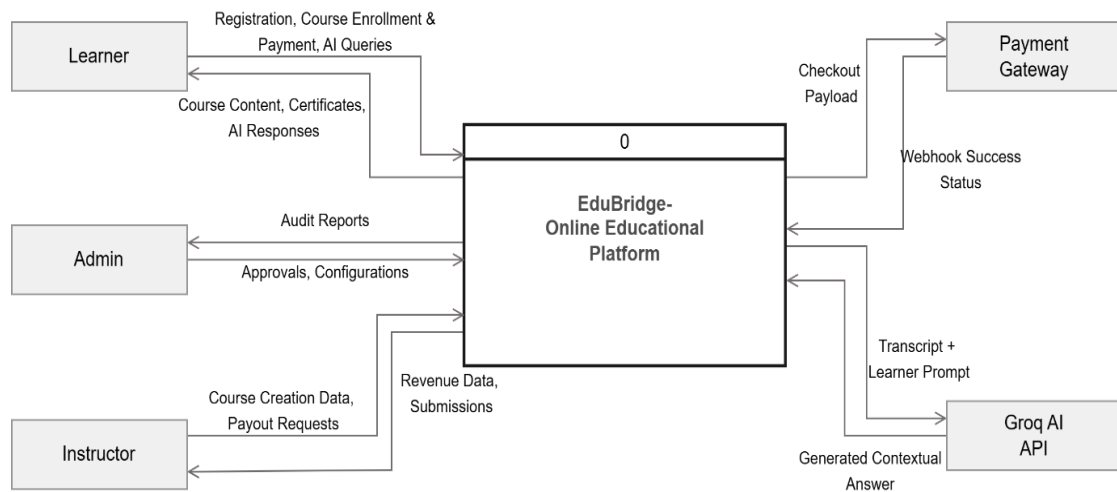


Figure 7.9: Level 0 Data Flow Diagram (Context Diagram) of the EduBridge Platform.

7.2.2 Level 1 Data Flow Diagram

Level 1 DFD breaks down the overall central process as shown in the Context Diagram into the key operational processes. At this level, the EduBridge system is split up into five separate

main processes; namely, User Management, Course Management, Enrollment and Payments, Interactive Learning and AI, and Assessments and Payouts. It is at this level that the Data Stores (D1 to D4) within the system are introduced, showing how data such as user information, lesson transcriptions, and payment transactions are permanently stored and shared among various processes.

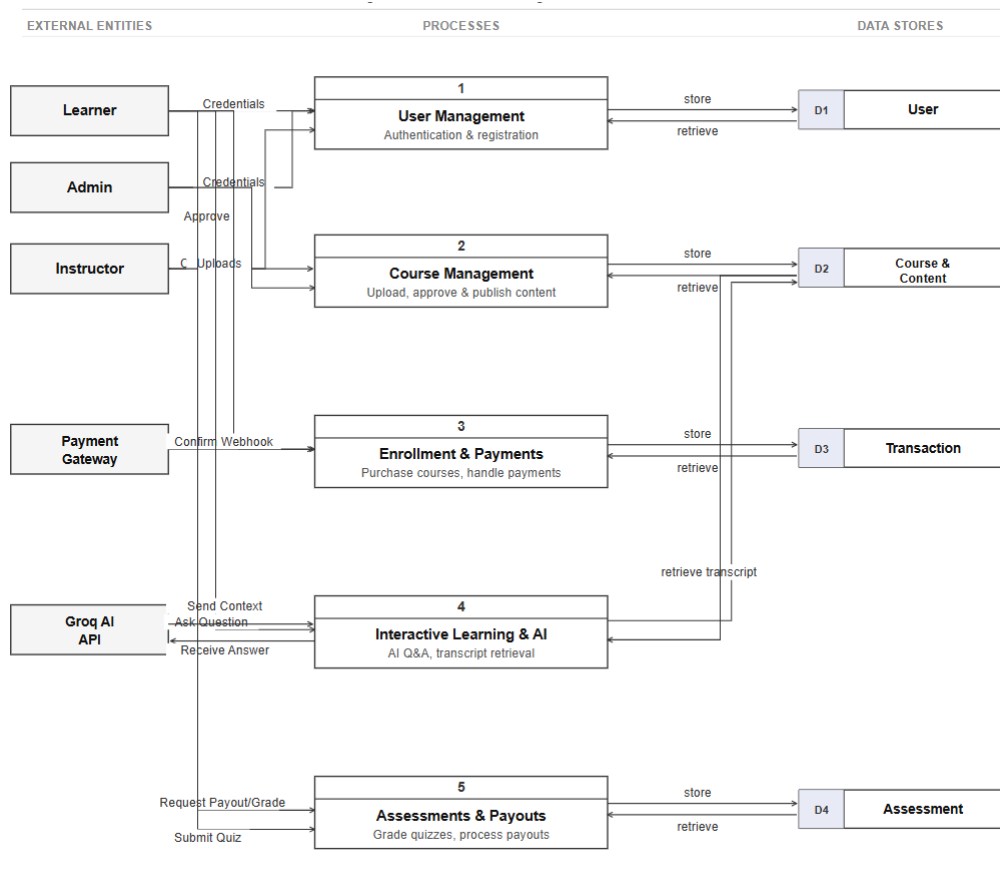


Figure 7.10: Level 1 Data Flow Diagram detailing core subsystems and data stores.

7.2.3 Level 2 Data Flow Diagrams

To get an in-depth analysis of the architecture of the system, the subsystems of EduBridge have been further divided in Level 2 Data Flow Diagrams. These diagrams focus on particular modules which show exactly what processes take place and how the database is involved in order to perform the business logic of the platform.

A. Process 1: User Management The following diagram depicts the flow of data for user management, profile updates, and "Teach with Us" instructor application moderation.

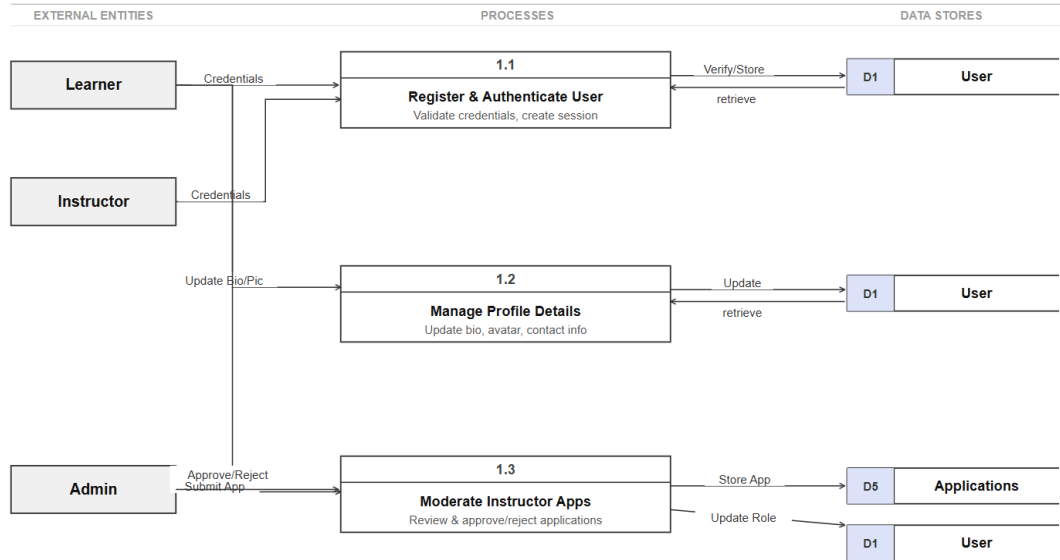


Figure 7.11: Level 2 DFD for Process 1.0 (User Management).

B. Process 2: Course Management The following figure depicts the process by which instructors upload metadata and media files, and administrators access the system in order to approve courses that are pending before publishing them to the course catalog.

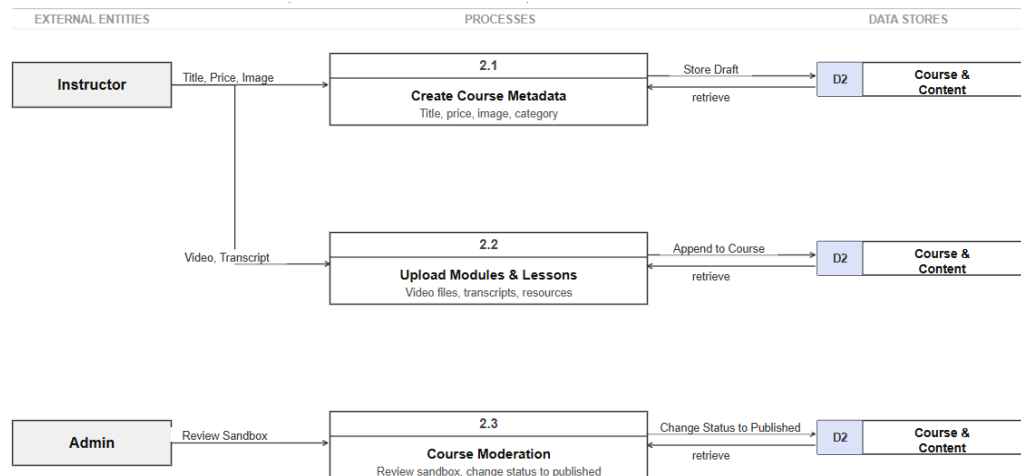


Figure 7.12: Level 2 DFD for Process 2.0 (Course Management).

C. Process 3: Enrollments & Payments This diagram illustrates the financial check out process flow where payments payload is generated by the system and external webhooks success notifications allow for database access to a student.

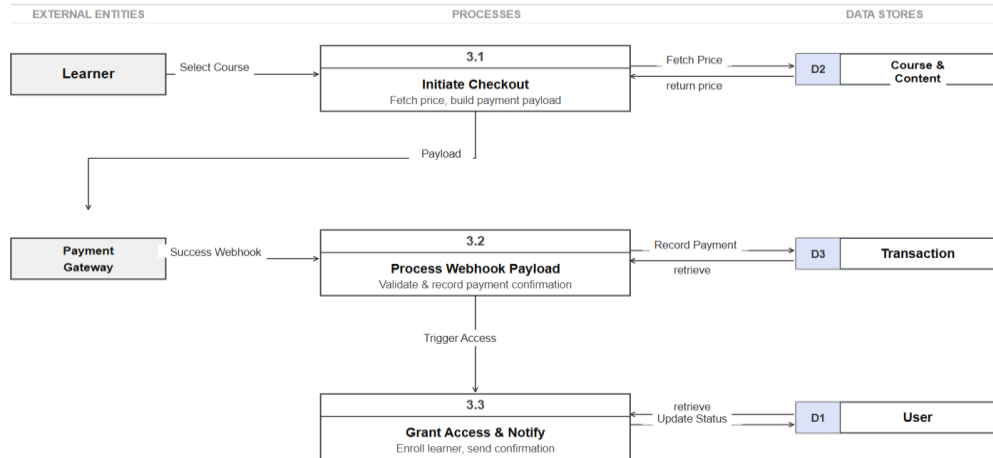


Figure 7.13: Level 2 DFD for Process 3.0 (Enrollments & Payments).

D. Process 4: Interactive Learning & AI This diagram shows the intricacy inside the RAG framework. Here is how a user's question gets combined with lesson transcripts inside the database before passing them through to the external Groq API to produce relevant answers.

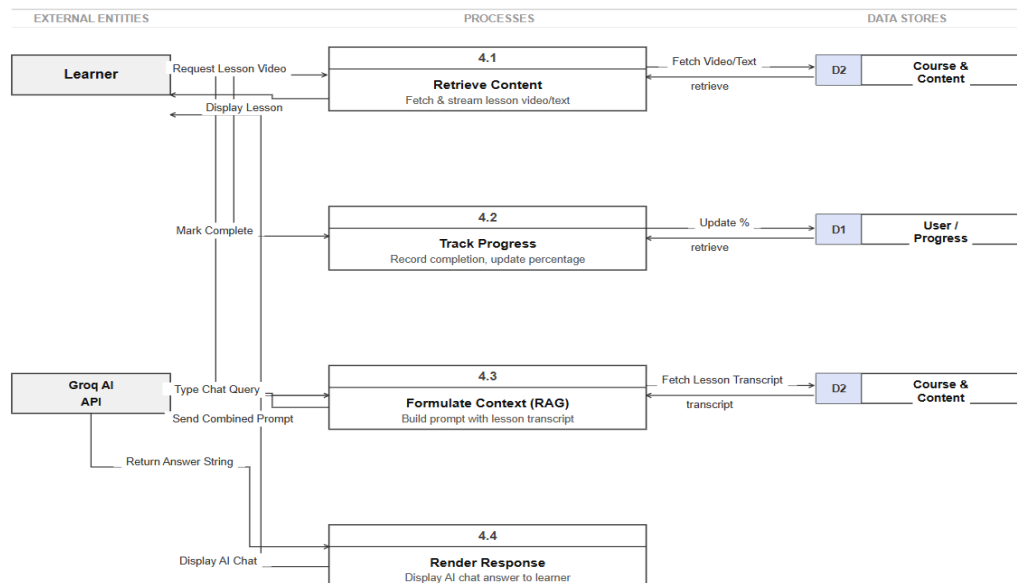


Figure 7.14: Level 2 DFD for Process 4.0 (Interactive Learning & AI Tutor Workflow).

E. Process 5: Assessments & Payouts This diagram describes the workflow of both the academic process (submission & automatic grading of quizzes), as well as financial processes that need to happen for teachers to claim their 70% cut of the earnings.

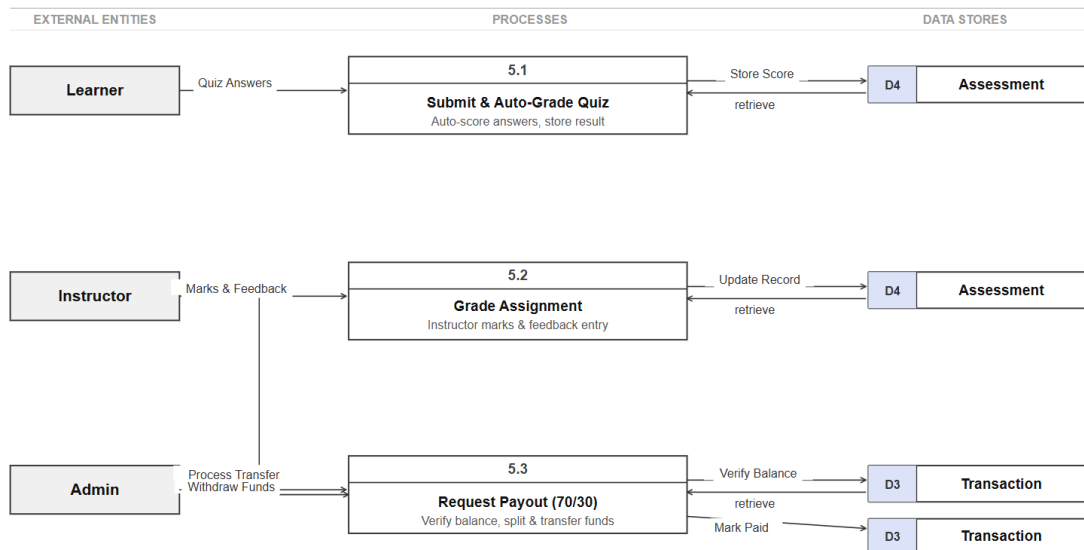


Figure 7.15: Level 2 DFD for Process 5.0 (Assessments & Financial Payouts).

7.3 Database Design

The database for the EduBridge application has been designed through the implementation of a strong RDBMS, i.e., the MySQL DBMS through the Laravel Eloquent ORM. The main aim of designing the database is to have very high integrity of the data, high security, and fast queries to be processed, especially during join operations for the RAG AI and financial payouts.

The database structure is completely relational in nature. Foreign Key constraints have been used throughout the schema to ensure referential integrity (e.g., the Enrollment cannot be there without User_ID and Course_ID). The entire structure has been mapped out using the UML Class Diagram (Refer to Chapter 4).

Table	Action	Rows	Type	Collation	Size	Overhead
announcements	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
answer_options	Browse Structure Search Insert Empty Drop	45	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
assignments	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
assignment_submissions	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
cache	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
cache_locks	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
categories	Browse Structure Search Insert Empty Drop	15	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
certificates	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
coupons	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
courses	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
enrollments	Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
instructor_applications	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
jobs	Browse Structure Search Insert Empty Drop	19	InnoDB	utf8mb4_unicode_ci	80.0 K	1 B
job_batches	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K	1 B
lessons	Browse Structure Search Insert Empty Drop	37	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
lesson_user	Browse Structure Search Insert Empty Drop	15	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
migrations	Browse Structure Search Insert Empty Drop	46	InnoDB	utf8mb4_unicode_ci	16.0 K	1 B
modules	Browse Structure Search Insert Empty Drop	20	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
notifications	Browse Structure Search Insert Empty Drop	38	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K	1 B
payout_requests	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
questions	Browse Structure Search Insert Empty Drop	12	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
quizzes	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 K	1 B
quiz_attempts	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
reviews	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
sessions	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
transactions	Browse Structure Search Insert Empty Drop	17	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
users	Browse Structure Search Insert Empty Drop	18	InnoDB	utf8mb4_unicode_ci	48.0 K	1 B
waitlists	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	64.0 K	1 B
30 tables	Sum	325	InnoDB	utf8mb4_general_ci	1.3 MiB	0 B

Figure 7.16: Database Schema

7.3.1 Normalization of Data Fields

Database normalization is the structured method of structuring data fields and tables to reduce redundancy and dependency. In the case of EduBridge system, all the tables are normalized up to the Third Normal Form (3NF).

For showing this process, the following tables show how a flat table of a purchase of a course by a student is sequentially decomposed to relational tables.

1. Normal Form (1NF)

The 1NF is achieved when the table does not have any repeating groups and all the attributes have atomic (not divisible) values. Every row in the table should be unique and identifiable using a Primary Key (Enrollment_ID in our case).

Table 7.1: Unnormalized / 1NF Course Enrollment Data

Enrollment_ID	Student_ID	Student_Name	Course_ID	Course_Title	Instructor_ID	Instructor_Name
ENR-101	U-001	Rhidi	C-50	React Native	I-09	Alamin Rahman
ENR-102	U-002	Rhidi	C-50	React Native	I-09	Alamin Rahman
ENR-103	U-001	Rhidi	C-88	Cyber Security	I-12	Maximilan

2. Normal Form (2NF)

In order to be in 2NF, the database needs to first be in 1NF, and all partial dependencies need to be eliminated. This implies that any non-primary key column should have dependency on the whole primary key. We solve this problem by segregating the transaction (Enrollment) from the other entities (Users and Courses).

Table 7.2: Enrollments Junction Table (2NF)

Enrollment_ID (PK)	Student_ID (FK)	Course_ID (FK)
ENR-101	U-001	C-50
ENR-102	U-002	C-50
ENR-103	U-001	C-88

Table 7.3: Students Table (2NF)

Student_ID (PK)	Student_Name
U-001	Rhidi
U-002	Rhidi

Table 7.4: Courses Table with Transitive Dependencies (2NF)

Course_ID (PK)	Course_Title	Instructor_ID	Instructor_Name
C-50	React Native	I-09	Alamin Rahman
C-88	Cyber Security	I-12	Maximilan

3. Normal Form (3NF)

In order to satisfy 3NF, we need to have 2NF first, and then get rid of all the transitive dependencies. From the above "Table 3: Courses," Instructor_Name is dependent on Instructor_ID, and not the primary key (Course_ID). The separation of the Instructor is required to eliminate redundancy.

Table 7.5: Normalized Courses Table (3NF)

Course_ID (PK)	Course_Title	Instructor_ID (FK)
C-50	React Native	I-09
C-88	Cyber Security	I-12

Table 7.6: Normalized Instructors Table (Mapped via users table)

Instructor_ID (PK)	Instructor_Name
I-09	Alamin Rahman
I-12	Maximilan

In achieving 3NF, any update to the user's name or course name in the EduBridge database is done through only one row in the database, making sure the data remains perfectly consistent.

7.3.2 Entity Relationship Model

Entity-Relationship Model acts as the architectural layout for the EduBridge database, showing how the database looks from a logical perspective. The ER diagram shows important entities such as Users, Courses, Quizzes, and Transactions among others, using the Chen's notation. It

also shows their respective attributes and primary keys. The most important thing about this diagram is that it establishes the cardinalities of the relationships among the different entities, like the one-to-many relation between the Instructor and his/her Courses and the many-to-many relation between the Learners and the Courses (through the enrollment entity).

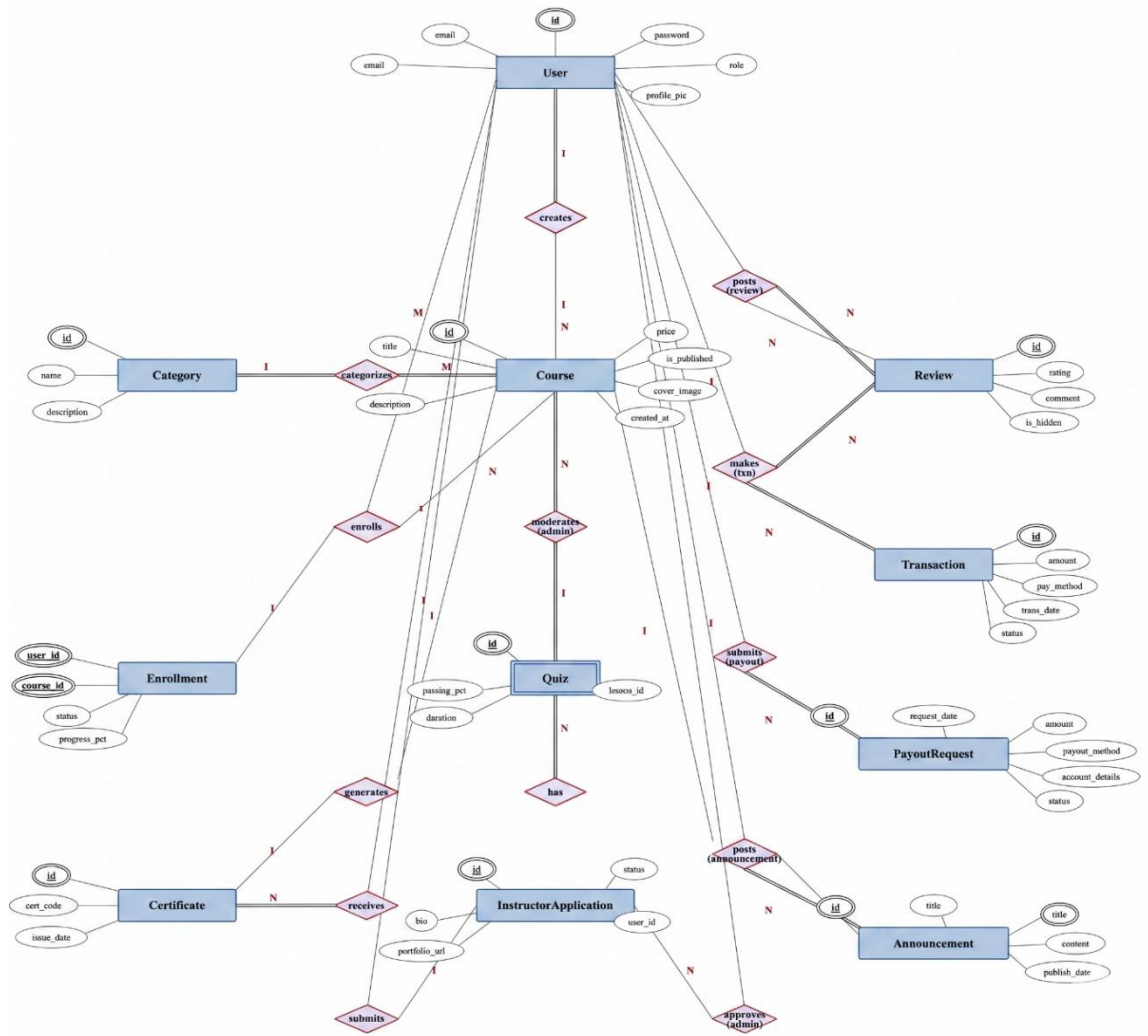


Figure 7.17: Entity Relationship Diagram (ERD) of the EduBridge Platform.

Chapter 8. System Quality and Testing

8.1 System Quality Management

System Quality Management (SQM) ensures that the EduBridge portal operates at the maximum performance, reliability, usability, efficiency, maintainability, portability, and security. With the consideration of financial gateways and third-party AI being involved in the portal, very strict quality measures have been put in place during its entire design phase. The following quality measures have been taken into consideration in the design of this portal:

A. Functionality:

- The system reliably performs all desired functions, which include:
- Secure registration and authentication for learners and instructors.
- Browsing through course listing, filtering, and video content provision.
- Revenue split (70/30) and secure payment processing via Stripe/SSLCommerz.
- Retrieval-Augmented Generation (RAG) AI tutor answer with the use of Groq API.
- Automatic grading of quizzes and creation of dynamic PDF certificate.
- All modules have been extensively tested to function perfectly.

B. Reliability:

- The system is able to operate optimally even when used simultaneously by multiple users and streaming videos.
- The database operations ensure that the data is reliable, especially reversing the enrollment transaction in case the payment webhook fails.
- The API service classes are designed to ensure fault tolerance to avoid system failures in case the external Groq AI engine fails.

C. Usability:

- The UI is made using Tailwind CSS, making the application user-friendly, efficient and distraction-free.
- The role-based dashboard (learner, instructor, admin) keeps the functionalities organized in such a way that users can easily navigate through them.

- Convincing feedback is provided to the user through validation messages, loading state and toast notification messages.

D. Efficiency:

- The asynchronous Laravel Queue workers take care of the intensive background task (such as video compression, email broadcasting, PDF generation etc) to avoid freezing of user interface.
- Meanwhile, the efficient indexing of MySQL makes the data fetching quick even as the catalog of courses and the number of users increase.

E. Maintainability:

- The application strictly adheres to the MVC design pattern.
- The separation of the front-end interface and back-end REST API helps in debugging and future feature addition.
- Clear and detailed inline documentation is provided along with good folder structure.

F. Portability:

- The application is developed using cross-platform web technologies.
- The web application is fully responsive according to different view port sizes of desktop, tablet and mobile devices without installing any native app.

G. Security:

- User password is cryptographically hashed using Bcrypt.
- API endpoints are secured by employing strong middleware and session tokens to protect from any unauthorized access.
- Webhooks for financial transactions have cryptographic signature checks to prevent enrollment spoofing frauds.
- Input sanitization is employed to prevent cross-site scripting and SQL injections.

H. Testing Approach:

- Unit Testing: Performed for isolated backend logic such as revenue share algorithm and auto-grading math.
- Integration Testing: To ensure seamless integration among the core system, Stripe API and Groq AI engine.
- User Acceptance Testing (UAT): Carried out on the beta tester end to validate course creation and contextual answering of AI Tutor.

I. Quality Assurance Methods:

- Code review by peers before the merge of the feature into production branch.
- Proper version control management with the help of Git and GitHub.
- Multiple tests for edge cases such as upload of corrupted file or exceeding API call limit.

8.2 Quality Management Process

Quality Management within the EduBridge project aims at ensuring that the final software is of a high standard both academically and commercially as an e-learning application. This process involves three stages; namely, Quality Planning, Quality Assurance and Quality Control.

A. Quality Planning:

This involves identifying the standard and methodology used from the beginning.

- Objectives: Creating a highly secure, AI-enabled, and scalable education platform.
- Activities: Identifying the best technology stack (Laravel, Tailwind, MySQL) and creating a systematic way to test webhooks.

B. Quality Assurance (QA):

Defect prevention is a key objective during the actual coding.

- Objectives: Adherence to architectural rules in order to avoid errors when testing.

- Activities: Maintaining discipline within GitHub through version control, static code analysis using PHP, and proper vaulting of environment variables (API keys).

C. Quality Control (QC):

This stage is about intense testing of the compiled system to identify and fix any problems found during this process.

- Objectives: To identify any bugs, evaluate API latency, and fulfill non-functional requirements.
- Activities: * Testing the manual workflow (such as Draft Course > Admin Review > Publish).
- AI boundary testing to make sure that the RAG tutor can only answer questions from the transcript of the lesson.
- Penetration testing (testing for any weaknesses in the Stripe payment system) to find ways to circumvent the checkout in order to get unauthorized access to the course.
- Checking cross-browser compatibility and responsiveness of the UI.

D. Continuous Improvement:

For future scalability, the system design allows for iterative updates.

- Practices: Logging errors centrally (Laravel Telescope) to identify slow queries and delivery failures of webhooks, so that developers can apply timely hotfixes.

E. Tools Used for Quality Management:

Table 8.1: Tools and Technologies Utilized for Quality Assurance

Tool	Purpose
GitHub	Version control, branch management, and collaboration
PHPUnit	Executing automated unit and feature tests for backend controllers and database models

Chrome DevTools	Inspecting DOM elements, debugging frontend interactivity, and testing Tailwind CSS responsiveness
Laravel Telescope	Real-time monitoring of database queries, mail, and queued jobs

8.3 Software Test Cases

A test case involves a collection of conditions put together to check whether or not a particular function in software performs in accordance with its technical specifications. The test cases carried out on the EduBridge platform are presented below.

Table 8.2: Test Case 1 - Secure Course Enrollment

Test Case ID: 1		Test Priority: High			
Module Name:		Enrollment & Payments			
Test Title:		Stripe Webhook Enrollment Verification			
Test Designed Date:		2026-03-15			
Description:		Ensures a user is only granted course access upon a verified successful payment payload from the gateway.			
Pre-Condition:		User is logged in; Course exists and is priced > 0.			
Dependencies:		Stripe API, Internet Connection			
Steps	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Click "Enroll Now"	User_ID: 5, Course_ID: 10	Redirected to Stripe Checkout	Redirected to Checkout	Pass
2	Complete Payment	Valid Test Credit Card	Payment clears, Webhook sent to backend	Webhook received and validated	Pass
3	Check Database	transactions table	Status="Completed", 70/30 split calculated	Split calculated accurately	Pass
4	Verify Access	enrollments table	Status="Active", Course unlocked in UI	Course viewer accessible	Pass

Table 8.3: Test Case 2 - AI Tutor Contextual Accuracy (RAG)

Test Case ID: 2		Test Priority: High			
Module Name:		Interactive Learning (AI)			
Test Title:		RAG Contextual Confinement Test			
Test Designed Date:		2026-03-18			
Description:		Ensures the Groq AI Tutor answers questions strictly based on the provided lesson transcript, preventing irrelevant hallucinations.			
Pre-Condition:		User is enrolled in a course; Lesson contains a valid text transcript.			
Dependencies:		Groq API			
Steps	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Open Course Viewer	Lesson: "Intro to React"	Video and Chat load successfully	Viewer loads correctly	Pass
2	Send Prompt (On Topic)	"What is a React Hook?"	AI returns answer referencing the transcript	Accurate, contextual answer provided	Pass
3	Send Prompt (Off Topic)	"Who won the World Cup?"	AI politely declines, citing lack of context in lesson	AI refuses to answer off-topic prompt	Pass

Table 8.4: Test Case 3 - Instructor Revenue Payout

Test Case ID: 3		Test Priority: High			
Module Name:		Financial Management			
Test Title:		Payout Request & Validation			
Test Designed Date:		2026-03-22			
Description:		Verifies that an instructor cannot request more funds than their calculated 70% share available balance.			
Pre-Condition:		Instructor account has a balance of \$100.00.			
Dependencies:		Admin Approval Dashboard			
Steps	Test Steps	Test Data	Expected Result	Actual Result	Status

1	Enter Payout Amount	Amount: \$150.00	System rejects request (Insufficient Funds)	Validation error shown to user	Pass
2	Enter Valid Amount	Amount: \$50.00	Request created, status set to "Pending"	Request successfully saved	Pass
3	Admin Review	View Admin Dashboard	Admin sees pending \$50.00 request	Request visible to Admin	Pass
4	Admin Approves	Click "Mark Paid"	Balance drops to \$50.00, Status="Paid"	Database updates correctly	Pass

Table 8.5: Test Case 4 - Automated Quiz Grading

Test Case ID: 4		Test Priority: Medium			
Module Name:		Assessments			
Test Title:		Auto-Grade Calculation			
Test Designed Date:		2026-03-25			
Description:		Verifies the system accurately calculates the percentage of correct answers and assigns a Pass/Fail flag.			
Pre-Condition:		Quiz has 4 questions; Passing threshold is 75%.			
Dependencies:		None			
Steps	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Select Answers	3 Correct, 1 Incorrect	Selections temporarily stored in state	UI reflects selections	Pass
2	Click "Submit Quiz"	Button Click Event	System evaluates answers against DB	Calculation initiated	Pass
3	Evaluate Score	Math Calculation	Total Score = 75%	Score exactly 75%	Pass
4	Assign Flag	Passing logic	75% >= 75%, Status = "Passed"	Status correctly set to Passed	Pass

Table 8.6: Test Case 5 - Course Moderation Workflow

Test Case ID: 5		Test Priority: High			
Module Name:		Content Management			
Test Title:		Draft to Published State Change			
Test Designed Date:		2026-03-28			
Description:		Ensures instructor courses are not visible to learners until an Administrator explicitly approves them.			
Pre-Condition:		Instructor has completed drafting a course.			
Dependencies:		None			
Steps	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Instructor Submit	Click "Submit for Review"	Course is_submitted = True	Status changes to Pending Review	Pass
2	Learner Catalog Check	Search for Course Title	Course does NOT appear in public search	Course successfully hidden	Pass
3	Admin Approval	Click "Publish Course"	Course is_published = True	Database triggers status change	Pass
4	Learner Catalog Check	Search for Course Title	Course now visible and purchasable	Course appears correctly	Pass

Chapter 9. Ethical Consideration

9.1 Ethical Considerations in the Software Development Process

The creation of EduBridge, an educational tool that will help in online learning and teaching, will entail dealing with personal student information, intellectual property of the teachers, financial transactions and Artificial Intelligence. This means that ethical responsibility is critical since it will determine the manner in which the system works.

9.1.1 Data Privacy and Security

It works with very confidential information such as user login credentials, personal profiles of students and teachers, and payout information. In order to ensure data confidentiality, all user passwords are hashed with Bcrypt algorithm. Moreover, in case of conducting financial transactions, they are outsourced to a PCI-compliant gateway (Stripe and SSLCommerz). Thus, no user payment information is ever stored locally on our servers. Besides, while using Groq AI API service for our interactive tutor functionality, we apply RAG (Retrieval-Augmented Generation) technology and provide our service with lesson transcript and user input. Any personally identifiable information is removed from it to avoid any potential data leakage to the AI models.

- **Intellectual Property:** Within the scope of the online learning and teaching system, Intellectual Property (IP) is one of the fundamental requirements to adhere to. For starters, the system ensures that professors will always be the absolute owners of the video courses that they will upload into the system. Second, all codes and documentation created during the process of development are original unless stated otherwise. Third, third-party applications and plugins such as Laravel and Tailwind are free and properly licensed. Fourth, plagiarism from external education materials is prohibited.
- **User Impact:** The design aims at delivering a positive and empowering experience for its users. The user experience is created in such a way that there are no dark patterns used in the learner's interface – they won't make students buy something they don't need or want. The instructors have their income clearly marked on the screen – the money distribution in 70/30% ratio makes them know how much they

earn from this platform. All the errors that may appear will be clear and unobtrusive.

- **Bias and Fairness:** All the activities carried out within EduBridge will be equitable in nature. Equal access will be offered to all learners with respect to the courses and AI tutors. It is important to point out that the AI tutor is limited through prompting such that it gives answers based only on lesson transcripts. The RAG method will ensure that the AI does not hallucinate or introduce other types of bias into learning material. Automated quizzes are graded in accordance to strict math formulas.
- **Responsibility to Stakeholders:** This involves the consideration of needs of all stakeholders of the platform – from those requiring good educational services to the tutors who deserve fair wages as well as those taking care of the integrity of the platform. Testing is done to ensure the delivery on the promise to ensure payments are done on time, and genuine certificates awarded. The developers are responsible for the accuracy and consistency of the platform, which earns 30%.
- **Professional Responsibility:** All stages involved in creating this e-learning and e-teaching application strictly observe ethical conduct as prescribed in the ACM Code of Ethics. These include providing software of the highest quality and reliability, being truthful concerning any shortcomings within the application (for instance, clearly stating that the AI Tutor is a virtual assistant rather than a live tutor), safeguarding user information, and using technology in ways that promote the education industry.

9.2 Sustainability in the Software Development Process

Sustainability is a critical consideration in the architecture of this online learning and teaching platform to ensure its long-term viability, affordability, and positive societal impact.

- **Economic Sustainability:** It is designed in a manner that makes it economically viable for those who operate it and the users. This is because it uses cheap technologies such as PHP/Laravel, MySQL, which are also open-source technologies and scalable cloud hosting, hence minimizing the overheads incurred. With the platform automatically taking a 30% commission from all course sales,

there will be enough income generated to cover the costs of maintaining the servers and using APIs such as Groq without any external funding being required.

- **Social Sustainability:** EduBridge promotes social sustainability by opening doors to educational opportunities for all members of society. EduBridge provides an online platform where students who are located far away from any educational institutions have a chance to take part in classes offered by experienced tutors. Moreover, EduBridge helps teachers become economically independent by giving them a tool that allows them to earn from teaching others their skills. Thus, the platform fosters lifelong learning and self-development.

Chapter 10. Conclusion

10.1 Brief Overview of the Project

EduBridge was conceived, designed, and built from scratch as a fully-featured online education and training portal. The main objective of the project is to link the best teachers with the most motivated students using a highly effective digital platform. It is developed with highly advanced technologies like Laravel (PHP), which is used as a back-end platform that can manage user authentication, creation of courses, distribution of multimedia and automated assessment. Besides being a typical e-learning platform, it has advanced features that go far beyond the limits of an ordinary e-learning platform. Some of these features include automated money splitting via Stripe and SSLCommerz webhooks, and the use of a RAG (Retrieval-Augmented Generation) AI Tutor with Groq API.

10.2 Proposed System Benefits

The introduction of the EduBridge platform has its own distinct advantages for stakeholders:

- **Democratized learning environment:** The system provides users with access to learning resources, quizzes, and artificial intelligence-based learning assistance 24/7.
- **Monetary independence of educators:** Thanks to the defined revenue share ratio, independent educators have an opportunity to use the platform to sell their services without concern over infrastructural and transactional issues.
- **The Personalized Learning Support System:** As a result of the Groq AI integration, it will be impossible for students to find themselves in a situation where they cannot go any further due to the difficulty of the topic. Instead, they will receive personalized assistance based on the course offered by an instructor.
- **Data safety:** Thanks to the database normalization and robust MVC architecture and webhooks management system, the system is both safe and transparent.

10.3 Limitations of the Project

Though the system is capable of meeting its main objectives, there is a need to point out its shortcomings in terms of this particular academic timeline:

- **Dependency on External APIs:** The payment processing and AI tutoring services of the system use external APIs, including Stripe, SSLCommerz, and Groq. Thus, the availability or changes in policies regulating the use of these services affect the functionality of the system.
- **Lack of Native Mobile Application:** There are no native iOS and Android apps developed for the system although the web application is designed for mobile use. This shortcoming results in the impossibility of downloading videos and push notifications.
- **AI Context Constraints:** The RAG AI Tutor cannot understand visual content in the lecture as well as mathematical formulas in the graph form.

10.4 Practicum and Its Value

It is hard to exaggerate the importance of this practicum even more so that it stands out to be the climax of the Computer Science program at the undergraduate level. From this perspective, the practicum served as the key link between theoretical computer science and practical software engineering.

In order to build EduBridge, we were required to apply all aspects of the Software Development Life Cycle (SDLC). It implied having the ability to normalize the database (3NF), modeling systems with the use of UML and Data Flow Diagrams (DFD). Apart from that, there was also a need to have an estimation of the cost of developing a project (function point analysis). Most importantly, we got a chance to deal with practical architectural issues, such as user authentication, relational databases, and MVC architecture via Laravel.

10.5 Future Plan

For the growth of EduBridge from being an academic project to an advanced educational software covering the whole world, the following alterations are supposed to occur in the future:

- Native mobile development: Developing independent applications for mobile devices through platforms like React Native or Flutter which will allow for offline learning and better user experience when taking courses via mobile phones.
- Localization and inclusion of the Bangla language: Increasing the number of languages used by the product and adding special elements of the Bangla user interface in addition to the localization of speech-to-text engines to provide for the needs of the target audience.
- Smart subjective assessment: Making the product's assessment engine analyze not only objective but also subjective tasks by means of NLP algorithms to avoid correction of essays done by teachers.
- Live webinars: Introducing the ability to conduct live classes in addition to video recordings in the courseware.

References

- Albrecht, A. J. and Gaffney, J. E. (1983) ‘Software function, source lines of code, and development effort prediction: a software science validation’, *IEEE Transactions on Software Engineering*, 9(6), pp. 639-648.
- Elmasri, R. and Navathe, S. B. (2015) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
- Groq (2026) *Groq API Documentation*. Available at: <https://console.groq.com/docs> (Accessed: 02 May 2026).
- Holmes, W., Bialik, M. and Fadel, C. (2019) *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston: Center for Curriculum Redesign.
- Laravel (2026) *Laravel - The PHP Framework for Web Artisans*. Available at: <https://laravel.com/docs> (Accessed: 15 April 2026).
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) ‘Retrieval-augmented generation for knowledge-intensive NLP tasks’, *Advances in Neural Information Processing Systems*, 33, pp. 9459-9474.
- Moore, J. L., Dickson-Deane, C. and Galyen, K. (2011) ‘e-Learning, online learning, and distance learning environments: Are they the same?’, *The Internet and Higher Education*, 14(2), pp. 129-135.
- Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Available at: <https://scrumguides.org/> (Accessed: 10 May 2026).
- Sommerville, I. (2015) *Software Engineering*. 10th edn. London: Pearson.
- Tailwind Labs (2026) *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. Available at: <https://tailwindcss.com/docs> (Accessed: 20 April 2026).

